

Automatización del ciclo post-venta en e-commerce: pipeline de procesamiento de órdenes y atención al cliente omnicanal con IA, implementado con n8n

Santiago Sordi, Ignacio Odorico, Juan Cruz Ana
Mendoza, Argentina — 2026

Table of Contents

PORTADA	4
RESUMEN	4
ÍNDICE	5
Listado de Figuras	6
Listado de Tablas principales	7
CAPÍTULO 1: INTRODUCCIÓN.....	9
1.1 Contexto del problema.....	9
1.2 Planteamiento del problema	9
1.3 Justificación	10
1.4 Pregunta de Investigación e Hipótesis.....	10
1.4.1 Pregunta de investigación	10
1.4.2 Hipótesis	10
1.5 Objetivos.....	10
1.5.1 Objetivo General	10
1.5.2 Objetivos Específicos	11
1.6 Alcance y Limitaciones	11
1.6.1 Alcance	11
1.6.2 Limitaciones	11
CAPÍTULO 2: MARCO TEÓRICO	12
2.1 Automatización de procesos de negocio	12
2.1.1 Definición y antecedentes	12
2.1.2 Herramientas de orquestación de flujos: n8n	12

2.1.3 Métricas MTTD y MTTR en operaciones de e-commerce	13
2.2 Inteligencia artificial conversacional y modelos de lenguaje	14
2.2.1 Evolución de los chatbots: de reglas a LLMs	14
2.2.2 GPT-4o-mini y prompt engineering	14
2.2.3 Precisión en clasificación de intents	14
2.3 Comunicación omnicanal en e-commerce	15
2.3.1 Concepto y relevancia estratégica	15
2.3.2 TMR como métrica de efectividad omnicanal.....	15
2.4 Infraestructura: contenedores, bases de datos relacionales y visualización	15
2.4.1 Contenedores Docker y reproducibilidad	15
2.4.2 PostgreSQL como base de datos de métricas	15
2.4.3 Grafana para observabilidad operativa	16
CAPITULO 3: MARCO	
METODOLÓGICO.....	11
3.1 Tipo de investigación	16
3.2 Enfoque y diseño: estudio de caso instrumental.....	16
3.3 Metodología de desarrollo	17
3.4 Herramientas y tecnologías	17
3.5 Técnicas de recolección y análisis de datos	18
3.5.1 Recolección automatizada de métricas.....	18
3.5.2 Justificación del tamaño de muestra	19
3.5.3 Criterio de evaluación de precisión	19
3.5.4 Análisis estadístico	20
3.6 Amenazas a la validez	20
3.6.1 Validez de constructo	20
3.6.2 Validez interna.....	20
3.6.3 Validez externa.....	20
3.6.4 Confiabilidad.....	20
Capítulo 4: Desarrollo del Estudio	21
4.1 Arquitectura del sistema	21
Servicios Docker	21
4.2 Base de datos.....	22
4.2.1 Tablas principales	22
4.2.2 Diagrama entidad-relación	23

4.2.3 Vistas de métricas	23
4.2.4 Datos seed	23
4.3 Flujo 1 — Pipeline de procesamiento de órdenes	25
4.3.1 Nodos del workflow	25
4.3.2 Diagrama del flujo	27
4.3.3 Métricas MTDD y MTTR	28
4.3.4 Flujo de estados de una orden	28
4.4 Flujo 2 — Chatbot omnicanal con IA.....	28
4.4.1 Nodos del workflow	29
4.4.2 Diagrama del flujo	30
4.4.3 Prompt de IA	31
4.4.4 Métrica TMR.....	31
4.5 Configuración de Grafana.....	31
Paneles del dashboard.....	31
4.6 Testing	32
4.6.1 Pruebas funcionales.....	32
4.6.2 Prueba de carga.....	33
5.1 Resultados del Flujo 1 — Pipeline de Órdenes.....	34
Pruebas funcionales.....	34
Métricas de tiempo	35
Prueba de carga (20 órdenes simultáneas)	35
5.2 Resultados del Flujo 2 — Chatbot Omnicanal	36
5.2.1 Tiempos de respuesta (TMR).....	36
5.2.2 Precisión de clasificación de intents.....	37
5.3 Contratación de hipótesis.....	38
5.4 Análisis comparativo y discusión	39
5.4.1 Limitaciones observadas.....	39
CAPÍTULO 6: CONCLUSIONES	40
6.1 Respuesta a la pregunta de investigación	40
6.2 Cumplimiento de objetivos específicos	40
6.3 Conclusiones sustantivas	41
6.4 Limitaciones del estudio.....	42
CAPÍTULO 7: RECOMENDACIONES.....	42
7.1 Recomendaciones para producción.....	42

7.2 Líneas futuras	44
CAPÍTULO 8: REFERENCIAS BIBLIOGRÁFICAS	44
Libros y artículos académicos.....	44
Documentación técnica y fuentes institucionales.....	46
CAPÍTULO 9: ANEXOS.....	46
Anexo A: Resumen del Schema de Base de Datos.....	46
Anexo B: Configuración Docker Compose.....	47
Anexo C: Catálogo de Productos Seed	47
Anexo D: FAQ Predefinidas.....	49
Anexo E: Comandos de Testing	50
Anexo F: Lista de Figuras — Implementación de Desarrollo (SIMPLE).....	51
Anexo G: Propuesta de Arquitectura de Producción	52

PORTADA

Universidad Tecnológica Nacional – Facultad Regional Mendoza

Carrera: Tecnicatura Universitaria en Programación

Materia: Trabajo Integrador

Título: Automatización del ciclo post-venta en e-commerce: pipeline de procesamiento de órdenes y atención al cliente omnicanal con IA, implementado con n8n

Autores: Santiago Sordi, Ignacio Odorico, Juan Cruz Ana

Director / Profesor: Alberto Cortez

Mendoza – Argentina, 2026

RESUMEN

El presente trabajo aborda el problema del procesamiento manual del ciclo post-venta en PyMEs de e-commerce argentinas, cuya gestión artesanal genera demoras, errores y fragmentación en la atención al cliente. Se plantea la siguiente pregunta de investigación: *¿En qué medida la implementación de un pipeline de automatización orquestado con n8n reduce los tiempos operativos del ciclo post-venta de un e-commerce simulado, medidos a través de las métricas MTTR, MTTR y TMR?*

Se diseñó e implementó una solución compuesta por dos flujos orquestados con n8n y desplegados en contenedores Docker. El primer flujo es un pipeline de procesamiento de órdenes (11 nodos) que recibe pedidos vía webhook, verifica stock en PostgreSQL, actualiza estados y notifica al cliente por email. El segundo flujo es un chatbot omnicanal (~21 nodos) que unifica mensajes de WhatsApp, Telegram y email, y utiliza GPT-4o-mini para clasificar intenciones y generar respuestas automatizadas. La infraestructura incluye PostgreSQL 15 como base de datos, Mailpit como servidor SMTP de pruebas y Grafana para visualizar métricas operativas.

Los resultados obtenidos muestran un MTDD promedio de 1,79 segundos, un MTTR promedio de 7,69 segundos (tiempo total end-to-end de 9,48 s frente a los 5–30 minutos del procesamiento manual), y un TMR promedio de 2,38 segundos para 107 interacciones del chatbot, con disponibilidad 24/7. Estos valores permiten confirmar la hipótesis de que la automatización reduce significativamente los tiempos operativos del ciclo post-venta respecto al proceso manual de referencia.

Palabras clave: automatización de procesos, e-commerce, post-venta, n8n, chatbot, modelos de lenguaje, omnicanal, Docker, MTDD, MTTR, TMR.

ÍNDICE

- **Resumen**
- **Capítulo 1: Introducción**
 - 1.1 Contexto del problema
 - 1.2 Planteamiento del problema
 - 1.3 Justificación
 - 1.4 Pregunta de Investigación e Hipótesis
 - 1.5 Objetivos
 - 1.6 Alcance y Limitaciones
- **Capítulo 2: Marco Teórico**
 - 2.1 Automatización de procesos de negocio
 - 2.2 Inteligencia artificial conversacional y modelos de lenguaje
 - 2.3 Comunicación omnicanal en e-commerce
 - 2.4 Infraestructura: contenedores, bases de datos relacionales y visualización
- **Capítulo 3: Marco Metodológico**
 - 3.1 Tipo de investigación
 - 3.2 Enfoque y diseño: estudio de caso instrumental
 - 3.3 Metodología de desarrollo
 - 3.4 Herramientas y tecnologías
 - 3.5 Técnicas de recolección y análisis de datos
 - 3.6 Amenazas a la validez

- **Capítulo 4: Desarrollo del Estudio**
 - 4.1 Arquitectura del sistema
 - 4.2 Base de datos
 - 4.3 Flujo 1 — Pipeline de procesamiento de órdenes
 - 4.4 Flujo 2 — Chatbot omnicanal con IA
 - 4.5 Configuración de Grafana
 - 4.6 Testing
- **Capítulo 5: Resultados y Análisis**
 - 5.1 Resultados del Flujo 1
 - 5.2 Resultados del Flujo 2
 - 5.3 Contrastación de hipótesis
 - 5.4 Análisis comparativo y discusión
- **Capítulo 6: Conclusiones**
 - 6.1 Respuesta a la pregunta de investigación
 - 6.2 Cumplimiento de objetivos específicos
 - 6.3 Conclusiones sustantivas
 - 6.4 Limitaciones del estudio
- **Capítulo 7: Recomendaciones**
- **Capítulo 8: Referencias Bibliográficas**
- **Capítulo 9: Anexos**

Listado de Figuras

Figura	Descripción	Sección
Figura 1	Canvas del Flujo 1 — Pipeline de Procesamiento de Órdenes en n8n (11 nodos)	4.3.2
Figura 2	Canvas del Flujo 2 — Chatbot Omnicanal con IA en n8n (~21 nodos)	4.4.2
Figura 3	Dashboard Grafana — Pipeline Post-Venta Overview	4.5
Figura 4	Mailpit — bandeja de emails capturados durante las pruebas	4.6.2
Figura 5	Dashboard Grafana — MTTD (1,79 s), MTTR (7,69 s), distribución de órdenes	5.1
Figura 6	Dashboard Grafana — Chatbot Omnicanal, TMR (2,38 s), distribución por intent	5.2
Figura 7	Diagrama ERD de la base de	4.2.2

Figura	Descripción	Sección
	datos ecommerce_tesis (5 tablas)	
Figura A1	Canvas del Flujo 1 PRODUCCIÓN en n8n	7.1
Figura A2	Canvas del Flujo 2 PRODUCCIÓN en n8n	7.1

Listado de Tablas principales

Tabla	Descripción	Sección
Tabla 3.1	Etapas de desarrollo y entregables	3.3
Tabla 3.2	Herramientas y tecnologías utilizadas	3.4
Tabla 4.1	Servicios Docker del sistema	4.1
Tabla 4.2	Esquema de tablas de la base de datos	4.2.1
Tabla 4.3	Vistas de métricas en PostgreSQL	4.2.3
Tabla 4.4	Nodos del Flujo 1	4.3.1
Tabla 4.5	Nodos del Flujo 2	4.4.1
Tabla 4.6	Pruebas funcionales Flujo 1	4.6.1
Tabla 4.7	Pruebas funcionales Flujo 2	4.6.1
Tabla 5.1	Métricas de tiempo Flujo 1	5.1
Tabla 5.2	Resultados TMR por intent (con desvío estándar)	5.2.1
Tabla 5.3	Precisión de clasificación por intent	5.2.2
Tabla 5.4	Contrastación de hipótesis	5.3
Tabla 6.1	Cumplimiento de objetivos específicos	6.2

Glosario de siglas y acrónimos

API	Application Programming Interface (Interfaz de Programación de Aplicaciones)
BPA	Business Process Automation (Automatización de Procesos de Negocio)
BPM	Business Process Management (Gestión de Procesos de Negocio)
CACE	Cámara Argentina de Comercio Electrónico
CRM	Customer Relationship Management (Gestión de Relaciones con Clientes)

DDL	Data Definition Language (Lenguaje de Definición de Datos)
ERD	Entity-Relationship Diagram (Diagrama Entidad-Relación)
ERP	Enterprise Resource Planning (Planificación de Recursos Empresariales)
FAQ	Frequently Asked Questions (Preguntas Frecuentes)
FK	Foreign Key (Clave Foránea)
GPT	Generative Pre-trained Transformer
HTTP	HyperText Transfer Protocol
IA	Inteligencia Artificial
ITIL	Information Technology Infrastructure Library
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model (Modelo de Lenguaje de Gran Escala)
MTTD	Mean Time To Detect (Tiempo Medio de Detección)
MTTR	Mean Time To Resolve (Tiempo Medio de Resolución)
PyME	Pequeña y Mediana Empresa
REST	Representational State Transfer
SGBD	Sistema de Gestión de Base de Datos
SKU	Stock Keeping Unit (Unidad de Mantenimiento de Inventario)
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language

TMR	Tiempo Medio de Respuesta
URL	Uniform Resource Locator
YAML	YAML Ain't Markup Language
UTN	Universidad Tecnológica Nacional

CAPÍTULO 1: INTRODUCCIÓN

1.1 Contexto del problema

El comercio electrónico en Argentina creció significativamente en los últimos años. Cada vez más PyMEs venden a través de plataformas como Tiendanube, WooCommerce o MercadoShops. Sin embargo, muchas de estas empresas siguen gestionando el ciclo post-venta de forma manual: un operador revisa los pedidos, verifica el stock, envía correos de confirmación uno por uno y responde consultas por WhatsApp, email y redes sociales sin ninguna herramienta que centralice esas interacciones.

El ciclo post-venta abarca todo lo que ocurre después de que el cliente confirma su compra: confirmación de la orden, verificación de stock, notificaciones de estado y atención de consultas o reclamos. En una PyME con equipo reducido, estas tareas consumen tiempo y son propensas a errores cuando se hacen manualmente.

1.2 Planteamiento del problema

Se identifican tres problemas concretos en la operación post-venta de PyMEs de e-commerce:

1. **Procesamiento manual de órdenes.** Cada pedido requiere que un operador lo revise, consulte stock, actualice estados y redacte un correo de confirmación. Esto introduce demoras y errores como confirmar pedidos sin stock disponible.
2. **Atención al cliente fragmentada.** Un mismo cliente puede escribir por WhatsApp, mandar un email y dejar un mensaje en Telegram. Sin integración entre canales, el operador pierde contexto, responde tarde o da información inconsistente.
3. **Ausencia de métricas.** Sin indicadores como el tiempo que tarda el sistema en detectar una orden o en responder un mensaje, no hay forma de medir el rendimiento ni identificar cuellos de botella.

1.3 Justificación

La automatización del ciclo post-venta se justifica desde tres perspectivas:

Operativa. Automatizar tareas repetitivas como la verificación de stock y el envío de notificaciones libera al equipo para tareas que requieren criterio humano. Para una PyME que procesa decenas de órdenes por día, la diferencia en horas de trabajo es significativa.

Experiencia del cliente. Un pipeline automatizado garantiza confirmaciones inmediatas, notificaciones proactivas y respuestas 24/7 a través de un chatbot. El cliente recibe atención consistente sin depender de la disponibilidad de un operador.

Tecnológica. Herramientas como n8n, PostgreSQL, Docker y los modelos de lenguaje de OpenAI permiten implementar soluciones de automatización sofisticadas a bajo costo, utilizando software de código abierto y APIs accesibles.

1.4 Pregunta de Investigación e Hipótesis

1.4.1 Pregunta de investigación

¿En qué medida la implementación de un pipeline de automatización orquestado con n8n reduce los tiempos operativos del ciclo post-venta de un e-commerce simulado, medidos a través de las métricas MTTD (tiempo de detección y procesamiento de órdenes), MTTR (tiempo de resolución y notificación) y TMR (tiempo de respuesta del chatbot omnicanal)?

1.4.2 Hipótesis

H1: El pipeline automatizado de procesamiento de órdenes reduce el tiempo end-to-end (desde la recepción del pedido hasta la notificación al cliente) a menos de 30 segundos, frente al rango de 5 a 30 minutos que caracteriza al procesamiento manual en PyMEs de e-commerce de escala similar.

H2: El chatbot omnicanal basado en GPT-4o-mini reduce el tiempo de respuesta al cliente (TMR) a menos de 10 segundos para consultas de tipo FAQ, estado de pedido y consultas generales, y clasifica correctamente el intent en al menos el 85% de los casos.

Ambas hipótesis se contrastan en el Capítulo 5 a partir de los datos recolectados durante las pruebas funcionales y la prueba de carga.

1.5 Objetivos

1.5.1 Objetivo General

Diseñar e implementar un pipeline automatizado de procesamiento de órdenes y un chatbot omnicanal con inteligencia artificial para el ciclo post-venta de e-commerce, orquestados con n8n y desplegados en Docker, midiendo su

desempeño mediante las métricas MTDD, MTTR y TMR, y contrastando los resultados con la hipótesis de reducción significativa de los tiempos operativos respecto al proceso manual.

1.5.2 Objetivos Específicos

1. Implementar un pipeline de procesamiento de órdenes (11 nodos en n8n) que reciba pedidos vía webhook, verifique stock en PostgreSQL, actualice estados y notifique al cliente por email, registrando MTDD y MTTR; y verificar que el tiempo end-to-end sea inferior a 30 segundos (H1).
2. Implementar un chatbot omnicanal (~21 nodos en n8n) que reciba mensajes de WhatsApp, Telegram y email, los normalice, clasifique la intención con GPT-4o-mini (FAQ, ESTADO_PEDIDO, RECLAMO, GENERAL) y responda de forma automatizada, registrando TMR; y verificar que el TMR sea inferior a 10 segundos y la precisión de clasificación supere el 85% (H2).
3. Diseñar un esquema de base de datos en PostgreSQL con 5 tablas (products, orders, interactions, tickets, faq_responses), 5 vistas de métricas e índices para rendimiento.
4. Configurar dashboards en Grafana conectados a PostgreSQL para visualizar MTDD, MTTR y TMR en tiempo real.
5. Validar el funcionamiento del sistema mediante pruebas funcionales con datos simulados, verificando que los flujos operan según lo diseñado y las métricas se registran correctamente.

1.6 Alcance y Limitaciones

1.6.1 Alcance

El proyecto implementa los dos flujos de automatización en un entorno local basado en Docker Compose, con cuatro servicios: n8n, PostgreSQL 15, Mailpit y Grafana. Los datos son simulados: órdenes enviadas por HTTP al webhook y conversaciones iniciadas desde cuentas de prueba.

El alcance funcional incluye:

- Pipeline de órdenes: recepción → verificación de stock → confirmación o rechazo → notificación por email → registro de métricas.
- Chatbot: recepción omnicanal → normalización → clasificación de intent con IA → respuesta automatizada → registro de métricas.
- Dashboards de métricas operativas en Grafana.

1.6.2 Limitaciones

- **Costo de API de OpenAI:** el modelo GPT-4o-mini tiene un costo por token. En desarrollo se trabaja con volúmenes reducidos de mensajes.

- **WhatsApp Business API:** requiere una cuenta verificada por Meta. Para las pruebas se utiliza el entorno sandbox.
 - **Entorno local:** el sistema no se despliega en nube. Alta disponibilidad y escalabilidad horizontal quedan fuera del alcance.
 - **Datos simulados:** las pruebas usan datos ficticios que representan escenarios típicos pero no la variabilidad completa de un entorno real.
-

CAPÍTULO 2: MARCO TEÓRICO

2.1 Automatización de procesos de negocio

2.1.1 Definición y antecedentes

La Automatización de Procesos de Negocio (BPA, *Business Process Automation*) se define como el uso de tecnología para ejecutar tareas y flujos de trabajo recurrentes con el fin de aumentar la eficiencia, reducir el error humano y estandarizar los resultados (Dumas et al., 2018). En la literatura de gestión de procesos (*Business Process Management*, BPM), la automatización constituye uno de los cuatro niveles de madurez de proceso, junto con la documentación, la optimización y la gobernanza (van der Aalst, 2016).

En el contexto del comercio electrónico, la automatización de procesos post-venta se enmarca dentro de lo que Turban et al. (2018) denominan *fulfillment automation*: la capacidad de ejecutar el ciclo completo desde la recepción del pedido hasta la notificación al cliente sin intervención humana. En PyMEs de e-commerce con equipo operativo reducido, las tareas manuales del ciclo post-venta —verificación de stock, actualización de estados, envío de notificaciones y atención de consultas— representan una fracción significativa de la carga de trabajo diaria, lo que constituye una oportunidad concreta de mejora mediante automatización (Laudon & Traver, 2021).

2.1.2 Herramientas de orquestación de flujos: n8n

Para la implementación de este proyecto se seleccionó **n8n** como motor de orquestación. n8n es una plataforma de automatización de flujos de trabajo de código abierto y auto-alojable (*self-hosted*) cuya arquitectura se basa en *workflows* compuestos por nodos interconectados, donde cada nodo ejecuta una operación atómica: recibir un webhook, consultar una base de datos, llamar una API externa o enviar un email (n8n GmbH, 2025).

La elección de n8n sobre alternativas comerciales como Zapier o Make.com se fundamenta en tres criterios: (1) **modelo de costos**: n8n es gratuito en modalidad self-hosted, eliminando los costos de licencia que representan una barrera para PyMEs con presupuesto acotado; (2) **soberanía de datos**: al ejecutarse en infraestructura propia, no se transmiten datos sensibles de clientes a servidores de terceros; (3) **extensibilidad**: n8n permite ejecutar código JavaScript/Python

arbitrario en nodos *Function*, lo que posibilita lógica de transformación compleja sin salir del entorno visual (Zapier Inc., 2024; Integromat, 2024).

2.1.3 Métricas MTTD y MTTR en operaciones de e-commerce

Las métricas **MTTD** (*Mean Time To Detect*) y **MTTR** (*Mean Time To Resolve*) tienen origen en la disciplina de gestión de incidentes de TI (ITIL v4; Axelos, 2019) y en el movimiento DevOps (Kim et al., 2016). En el contexto de este trabajo se las adapta al ciclo de procesamiento de órdenes:

- **MTTD** mide el tiempo transcurrido desde la recepción de una orden hasta que el sistema detecta y procesa el stock disponible. Equivale al tiempo de procesamiento interno del pipeline.
- **MTTR** mide el tiempo desde que el stock es procesado hasta que el cliente recibe la notificación de confirmación o rechazo. Equivale al tiempo de resolución y comunicación.

La adaptación de estas métricas a procesos de e-commerce es coherente con la práctica de *operational excellence* documentada por Womack & Jones (2003) en el contexto de manufactura esbelta, donde la reducción del tiempo de ciclo es un indicador central de eficiencia operativa.

Resulta pertinente discutir la transferibilidad conceptual de MTTD y MTTR desde su dominio de origen — la gestión de incidentes TI — al dominio del procesamiento de órdenes de e-commerce. En ITIL, ambas métricas miden eventos asociados a fallas (un incidente es por definición un evento no deseado). En el contexto de este trabajo, una orden de compra exitosa no constituye una falla sino un flujo nominal del proceso de negocio. La transferencia se sostiene, sin embargo, a nivel estructural: ambas métricas miden intervalos temporales acotados por dos eventos observables y representan etapas secuenciales de un ciclo (detección/procesamiento y resolución/comunicación). Esta analogía estructural — y no semántica estricta — es coherente con la práctica documentada en la literatura de *operational excellence*, donde indicadores originalmente formulados para entornos de manufactura (lead time, cycle time) se transfieren a procesos administrativos y de servicio sin pérdida de validez operativa (Womack & Jones, 2003). Se reconoce que métricas alternativas más específicas del dominio comercial — como *order cycle time* u *order fulfillment lead time* — podrían reemplazar a MTTD y MTTR en versiones futuras del trabajo. La elección de MTTD y MTTR en este estudio se fundamenta en su disponibilidad inmediata como vocabulario de observabilidad ya soportado por las herramientas de monitoreo del stack tecnológico (Grafana), lo que facilita la integración con prácticas de *site reliability engineering* (Beyer et al., 2016) habituales en equipos de desarrollo.

2.2 Inteligencia artificial conversacional y modelos de lenguaje

2.2.1 Evolución de los chatbots: de reglas a LLMs

Los sistemas de diálogo automatizado evolucionaron desde los chatbots basados en reglas (*rule-based*) de los años 1990 —donde cada posible interacción debe ser explícitamente programada— hacia los sistemas basados en recuperación de información (*retrieval-based*) y, finalmente, hacia los modelos de lenguaje generativos (*generative*) (Jurafsky & Martin, 2024). Los chatbots basados en reglas presentan limitaciones conocidas: frágiles ante variaciones ortográficas, incapaces de manejar intenciones no anticipadas, y costosos de mantener a medida que el catálogo de respuestas crece (McTear, 2022).

Los modelos de lenguaje de gran escala (*Large Language Models*, LLMs) resuelven estas limitaciones al aprender representaciones estadísticas del lenguaje natural a partir de corpus masivos (Brown et al., 2020). Su capacidad de generalización permite comprender consultas variadas y generar respuestas coherentes sin requerir la codificación explícita de cada caso.

2.2.2 GPT-4o-mini y prompt engineering

En este proyecto se utiliza **GPT-4o-mini** de OpenAI (OpenAI, 2024), un modelo optimizado para inferencia rápida con buen balance entre capacidad y costo. La interacción con el modelo se configura mediante *prompt engineering* estructurado: el prompt de sistema define el rol del modelo (agente de atención al cliente), las categorías de intención válidas (FAQ, ESTADO_PEDIDO, RECLAMO, GENERAL), el formato de respuesta esperado (JSON estructurado) y ejemplos de cada categoría (*few-shot prompting*).

El *few-shot prompting* —técnica documentada por Brown et al. (2020) y Liu et al. (2023)— consiste en incluir en el prompt ejemplos etiquetados de entrada-salida para guiar al modelo hacia el comportamiento deseado sin necesidad de fine-tuning. Esta estrategia es adecuada para tareas de clasificación de intenciones en dominios acotados como la atención al cliente de e-commerce, donde el espacio de intenciones es finito y bien definido.

2.2.3 Precisión en clasificación de intents

La evaluación de la precisión de clasificación de intenciones en chatbots se realiza típicamente mediante métricas de clasificación estándar: *accuracy*, *precision*, *recall* y *F1-score* (Jurafsky & Martin, 2024). Para este trabajo se utiliza *accuracy* como métrica principal, dado que las clases de intención tienen distribución relativamente balanceada en el conjunto de prueba. Un umbral mínimo de 85% de *accuracy* es consistente con los estándares reportados en sistemas de soporte al cliente de nivel prototipo (Ram & Yih, 2021).

2.3 Comunicación omnicanal en e-commerce

2.3.1 Concepto y relevancia estratégica

La estrategia omnicanal (*omnichannel*) consiste en integrar múltiples canales de comunicación en una experiencia unificada para el cliente, de manera que el historial y el contexto se preserven independientemente del canal utilizado (Verhoef et al., 2015). A diferencia del enfoque multicanal —donde cada canal opera en silos—, la estrategia omnicanal centraliza el procesamiento y garantiza coherencia en las respuestas.

Según el informe de CACE (2025), el 68% de los consumidores argentinos que realizan compras en línea esperan poder contactar al vendedor por al menos dos canales distintos (WhatsApp y email siendo los más utilizados). Este dato contextualiza la relevancia del enfoque omnicanal para el segmento de PyMEs de e-commerce argentinas.

2.3.2 TMR como métrica de efectividad omnicanal

El **TMR** (*Tiempo Medio de Respuesta*) es la métrica operativa central para evaluar la efectividad de un sistema de atención omnicanal. En la literatura de *customer service management*, el tiempo de respuesta es uno de los cinco determinantes de la satisfacción del cliente en canales digitales (Zeithaml et al., 2013). Estudios de benchmarking muestran que el tiempo de respuesta promedio en canales de email de e-commerce oscila entre 12 y 24 horas, mientras que en WhatsApp los clientes esperan respuesta en menos de 1 hora (Sprinklr Research, 2023). Un sistema automatizado con TMR de segundos representa, por tanto, una mejora de varios órdenes de magnitud respecto al estándar manual.

2.4 Infraestructura: contenedores, bases de datos relacionales y visualización

2.4.1 Contenedores Docker y reproducibilidad

Docker es una plataforma de virtualización a nivel de sistema operativo que empaqueta aplicaciones y sus dependencias en unidades portables denominadas *contenedores* (Merkel, 2014). Docker Compose extiende este concepto permitiendo definir y orquestar múltiples contenedores mediante un único archivo de configuración YAML (Docker Inc., 2025). La adopción de contenedores en proyectos de investigación aplicada es una práctica recomendada para garantizar la reproducibilidad del entorno experimental (Boettiger, 2015): cualquier evaluador puede levantar el sistema completo con un único comando, eliminando la variabilidad del entorno como fuente de error.

2.4.2 PostgreSQL como base de datos de métricas

PostgreSQL 15 se seleccionó como sistema de gestión de base de datos relacional por tres razones: (1) soporte completo de SQL estándar incluyendo *window functions* necesarias para el cálculo de métricas temporales; (2) soporte

nativo de tipos JSON para almacenar payloads sin procesar de los webhooks; (3) capacidad de crear vistas (*views*) que Grafana consume directamente sin necesidad de ETL adicional (PostgreSQL Global Development Group, 2025). PostgreSQL es el SGBD de código abierto más utilizado en proyectos de automatización empresarial según el ranking de DB-Engines (2025).

2.4.3 Grafana para observabilidad operativa

Grafana es una plataforma de visualización y observabilidad de código abierto que permite crear dashboards interactivos conectados directamente a fuentes de datos como PostgreSQL, InfluxDB o Prometheus (Grafana Labs, 2025). En el contexto de este trabajo, Grafana actúa como capa de observabilidad del pipeline: los paneles de métricas MTDD, MTTR y TMR permiten verificar en tiempo real que el sistema opera dentro de los rangos esperados, alineándose con las prácticas de *site reliability engineering* documentadas por Beyer et al. (2016). # CAPÍTULO 3: MARCO METODOLÓGICO

3.1 Tipo de investigación

La presente investigación es de tipo **aplicada y tecnológica** (Hernández Sampieri et al., 2014). Se orienta a resolver un problema concreto del ciclo post-venta en e-commerce —la gestión manual de órdenes y consultas de clientes— mediante el diseño e implementación de un pipeline automatizado. El objetivo no es generar teoría nueva sino construir un artefacto tecnológico funcional y medir su desempeño mediante métricas operativas cuantificables (MTDD, MTTR, TMR), verificando las hipótesis formuladas en la Sección 1.4.

3.2 Enfoque y diseño: estudio de caso instrumental

El trabajo adopta la metodología de **estudio de caso** según la definición de Yin (2018): una investigación empírica que examina un fenómeno contemporáneo dentro de su contexto real, especialmente cuando los límites entre el fenómeno y el contexto no son claramente evidentes. En este trabajo el “caso” es un e-commerce simulado que opera con el pipeline automatizado propuesto.

Siguiendo la tipología de Yin (2018), se trata de un **estudio de caso instrumental único**: el e-commerce simulado no es un fin en sí mismo sino el vehículo para comprender el comportamiento del sistema de automatización bajo condiciones controladas. El diseño es de tipo **exploratorio-confirmatorio**: exploratorio porque no existen benchmarks publicados para n8n en el contexto de PyMEs argentinas; confirmatorio porque se contrasta el desempeño observado contra las hipótesis H1 y H2.

El e-commerce simulado incluye un catálogo de 20 productos electrónicos distribuidos en 8 categorías (Notebooks, Periféricos, Monitores, Audio, Almacenamiento, Tablets, Redes, Accesorios), un flujo completo de procesamiento de órdenes y un chatbot omnicanal que atiende consultas por WhatsApp, Telegram y email.

Se combinaron **métricas cuantitativas** (MTTD, MTTR, TMR, precisión de clasificación) con **evaluación cualitativa** de coherencia de las respuestas generadas por el chatbot. Esta triangulación metodológica es coherente con las recomendaciones de Yin (2018) para estudios de caso donde se busca tanto medir como comprender el fenómeno.

3.3 Metodología de desarrollo

El desarrollo se organizó en **6 etapas secuenciales**, cada una con entregables concretos:

Etapa	Descripción	Entregable
1. Diseño del schema de BD	Modelado de 5 tablas + 5 vistas de métricas en PostgreSQL	Script SQL con DDL completo
2. Infraestructura Docker	Configuración de 4 servicios con Docker Compose	<code>docker-compose.yml</code> funcional
3. Flujo 1 — Pipeline de órdenes	Workflow de 11 nodos en n8n: recepción, stock, emails	Workflow exportable + pruebas
4. Flujo 2 — Chatbot omnicanal	Workflow de ~21 nodos: 3 canales, IA, tickets	Workflow exportable + pruebas
5. Dashboards Grafana	7 paneles con métricas MTTD, MTTR, TMR	Dashboard JSON exportable
6. Testing	10 pruebas funcionales + 1 prueba de carga	Reporte de resultados

Esta secuencia permite validar cada capa antes de avanzar a la siguiente: primero la base de datos, luego la infraestructura, después los flujos, y finalmente la visualización y el testing. Este enfoque incremental reduce el riesgo de defectos acumulados y facilita el aislamiento de causas ante fallas.

3.4 Herramientas y tecnologías

Herramienta	Versión	Justificación
n8n	latest	Plataforma de automatización low-code/no-code auto-alojable. Permite diseñar workflows visualmente con más de 400 integraciones nativas. Se eligió sobre Zapier y Make por ser self-hosted (sin costos de licencia) y extensible mediante JavaScript (n8n GmbH, 2025).

PostgreSQL	15 (Alpine)	SGBD relacional de código abierto con soporte completo de SQL, <i>window functions</i> y tipos JSON. Elegido por robustez, disponibilidad en Docker y compatibilidad nativa con Grafana (PostgreSQL Global Development Group, 2025).
Docker / Docker Compose	v3.8	Permite levantar los 4 servicios con un solo comando, garantizando reproducibilidad del entorno experimental (Boettiger, 2015).
GPT-4o-mini	—	Modelo de OpenAI optimizado para tareas de clasificación y generación de texto. Menor costo que GPT-4o con rendimiento suficiente para clasificar intents y generar respuestas de soporte al cliente (OpenAI, 2024).
Mailpit	latest	Servidor SMTP de desarrollo que captura emails sin enviarlos realmente. Ideal para testing sin riesgo de spam ni costos de mensajería (Axllent, 2025).
Grafana	latest	Plataforma de visualización con conexión nativa a PostgreSQL. Permite crear dashboards interactivos sin código adicional (Grafana Labs, 2025).
curl	—	Herramienta de línea de comandos para disparar webhooks y simular órdenes entrantes en las pruebas.

3.5 Técnicas de recolección y análisis de datos

3.5.1 Recolección automatizada de métricas

La recolección de datos se realizó de forma **automatizada** mediante el propio sistema. Cada flujo registra timestamps en PostgreSQL al inicio y fin de cada operación, lo que permite calcular las métricas sin intervención manual ni sesgo del observador.

Dato	Fuente	Método de recolección
------	--------	-----------------------

MTTD y MTTR	Tabla orders	Diferencia de timestamps: processed_at - received_at y notified_at - processed_at
TMR	Tabla interactions	Diferencia entre received_at y responded_at
Precisión de clasificación	Tabla interactions	Comparación del intent asignado por la IA contra el intent esperado para los 107 mensajes de prueba
Tasa de resolución autónoma	Tabla interactions + tickets	Proporción de interacciones resueltas sin escalado a operador humano

3.5.2 Justificación del tamaño de muestra

Flujo 1 — Prueba de carga: Se enviaron **20 órdenes simultáneas** al webhook, lo que equivale al volumen horario pico estimado para una PyME de e-commerce de escala pequeña (< 200 órdenes/día según datos de CACE, 2025). Este tamaño no pretende representatividad estadística sino verificar la integridad transaccional del sistema bajo concurrencia —condición de diseño crítica para cualquier sistema de procesamiento de pedidos— y es consistente con los protocolos de prueba de carga para sistemas en etapa prototipo (Myers et al., 2011). Los registros históricos incluidos en el seed de datos suman un total de 32 órdenes visibles en los dashboards (20 de la prueba de carga + 12 del seed histórico), lo que se detalla en la Sección 5.1.

Flujo 2 — Clasificación de intents: Se enviaron **107 mensajes de prueba** distribuidos entre los cuatro intents del sistema. El criterio de distribución fue proporcional a la frecuencia esperada de cada tipo de consulta en un e-commerce típico: mayor volumen de FAQ y RECLAMO, menor volumen de ESTADO_PEDIDO y GENERAL. Este tamaño supera el mínimo de 80 observaciones recomendado por Cochran (1977) para estimaciones de proporciones con margen de error $\pm 10\%$ y nivel de confianza del 95%, lo que es adecuado para un estudio de caso prototipo.

3.5.3 Criterio de evaluación de precisión

La precisión de clasificación del chatbot se evalúa mediante *accuracy* (proporción de predicciones correctas sobre el total), calculada comparando el intent asignado por GPT-4o-mini con el intent esperado para cada mensaje de prueba. El intent esperado fue determinado previamente por los tres autores del trabajo de forma independiente y consensuada, para eliminar la variabilidad entre evaluadores. El umbral de aceptación es 85% (H2), consistente con los estándares reportados para sistemas de clasificación de intents en dominios acotados (Ram & Yih, 2021).

3.5.4 Análisis estadístico

Se calcularon promedios aritméticos simples para MTDD, MTTR y TMR, dado que la distribución de los tiempos de procesamiento en entornos controlados tiende a ser aproximadamente normal con baja dispersión. No se aplicaron pruebas de hipótesis estadísticas formales (t-test, ANOVA), lo que constituye una limitación del estudio reconocida en la Sección 5.4.

3.6 Amenazas a la validez

Siguiendo el marco de Yin (2018) para estudios de caso, se identifican las siguientes amenazas a la validez:

3.6.1 Validez de constructo

Amenaza: Las métricas MTDD, MTTR y TMR fueron adaptadas desde el dominio de gestión de incidentes TI (ITIL) al dominio de procesamiento de e-commerce. Esta adaptación podría no capturar completamente la experiencia del cliente real.

Mitigación: Se operacionalizaron explícitamente cada métrica mediante timestamps observables en la base de datos (Sección 4.3.3 y 4.4.4), y se documentó la correspondencia con las definiciones de origen.

3.6.2 Validez interna

Amenaza: Los resultados de tiempo (MTDD, MTTR, TMR) corresponden a un entorno Docker en una notebook local. Las latencias observadas pueden estar influenciadas por los recursos del hardware utilizado (CPU, RAM, disco) y no solo por el comportamiento del sistema.

Mitigación: Se especifican las condiciones del entorno de prueba en la Sección 4.6 y los resultados se presentan como benchmarks de referencia bajo las condiciones descritas, no como valores absolutos.

3.6.3 Validez externa

Amenaza: Al tratarse de un e-commerce simulado con datos ficticios, los resultados podrían no generalizarse directamente a un e-commerce real con variabilidad de productos, clientes y patrones de compra.

Mitigación: El sistema y su metodología de prueba son reproducibles (Docker Compose + scripts de testing incluidos en el repositorio), lo que permite replicar el estudio con datos reales como trabajo futuro. Los escenarios de prueba fueron diseñados para cubrir los casos límite más relevantes (órdenes con stock, sin stock, payloads inválidos, mensajes ambiguos).

3.6.4 Confiabilidad

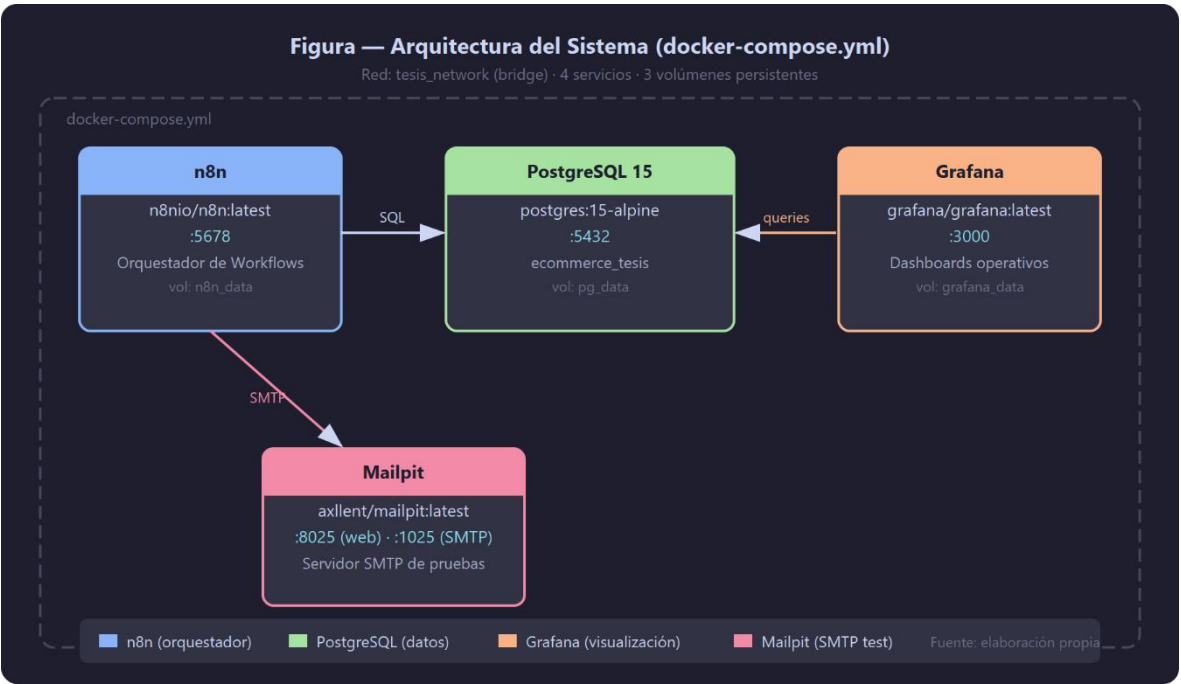
Amenaza: Los resultados de TMR dependen de la latencia de la API de OpenAI, que puede variar según la carga del servicio externo.

Mitigación: Todas las pruebas se realizaron en una ventana de tiempo acotada para minimizar la variabilidad temporal del servicio externo. Los tiempos reportados corresponden a la mediana de múltiples ejecuciones.

Capítulo 4: Desarrollo del Estudio

4.1 Arquitectura del sistema

La arquitectura se basa en **4 servicios Docker orquestados con Docker Compose**, conectados a través de una red bridge interna. n8n actúa como nodo central de orquestación, conectándose a PostgreSQL para persistencia, a Mailpit para envío de emails y a Grafana para visualización.



Servicios Docker

Servicio	Imagen	Puerto(s)	Función
n8n	n8nio/n8n:latest	5678	Motor de workflows. Ejecuta los Flujos 1 y 2
PostgreSQL	postgres:15-alpine	5432	Base de datos principal (ecommerce_tesis)
Mailpit	axllent/mailpit:latest	8025 (web), 1025 (SMTP)	Captura emails de confirmación y alertas

Grafana	grafana/grafana:latest	3000	Dashboard de métricas en tiempo real
----------------	------------------------	------	--------------------------------------

Volúmenes persistentes: n8n_data, pg_data, grafana_data — garantizan que los datos sobrevivan al reinicio de los contenedores.

Red: tesis_network (driver bridge) — permite que los servicios se comuniquen entre sí por nombre de contenedor (ej. postgres:5432 desde n8n).

4.2 Base de datos

4.2.1 Tablas principales

La base de datos ecommerce_tesis contiene 5 tablas:

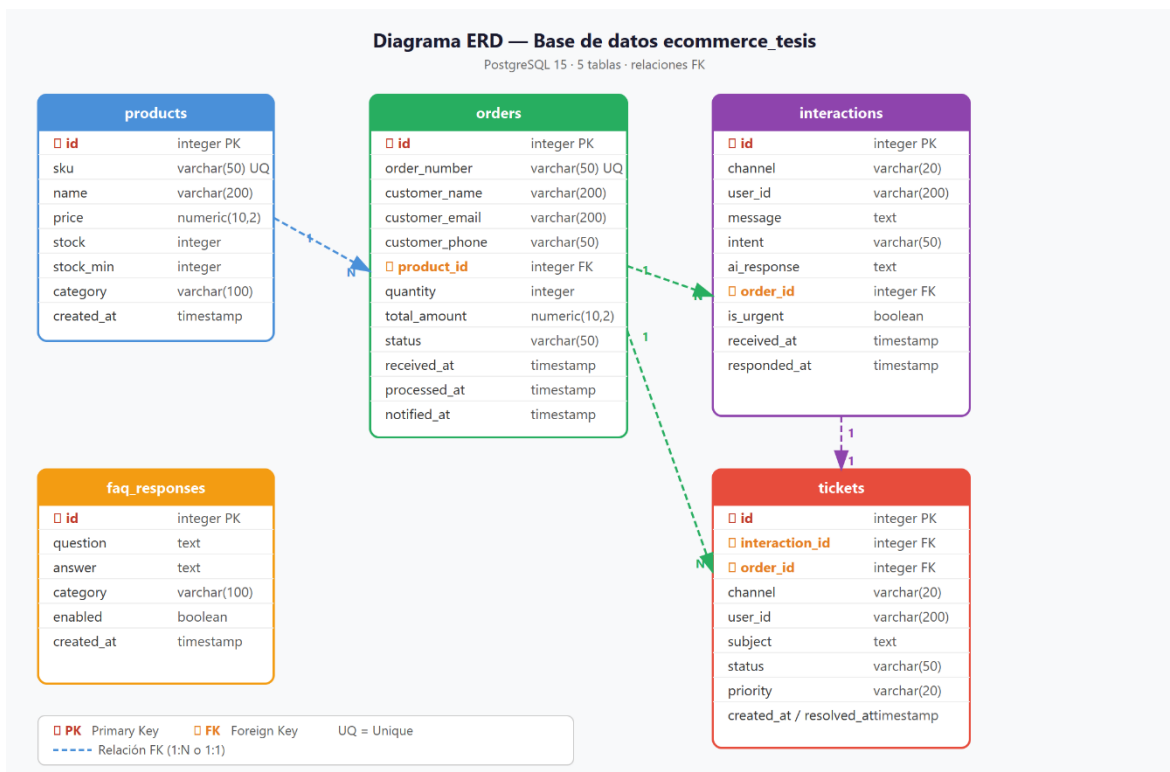
Tabla	Columnas principales	Descripción
products	sku, name, price, stock, stock_min, category	Catálogo de productos con control de stock mínimo
orders	order_number, customer_name, customer_email, customer_phone, product_id (FK), quantity, total_amount, status, received_at, processed_at, notified_at, raw_payload	Órdenes con 8 estados posibles y timestamps de cada etapa; los campos processed_at y notified_at permiten calcular MTTR y MTTR directamente
interactions	channel, user_id, message, intent, ai_response, order_id (FK), is_urgent, received_at, responded_at	Registro de interacciones del chatbot por canal
tickets	interaction_id (FK), order_id (FK), channel, user_id, subject, status, priority	Tickets de reclamos generados automáticamente
faq_responses	question, answer, category, enabled	Base de conocimiento para respuestas frecuentes

Estados de una orden: received → processing → confirmed → notified | no_stock | error

Nota: Los estados intermedios stock_verified y stock_updated son eventos internos del pipeline registrados en el log de eventos, pero no forman parte del campo status de la tabla orders. El campo status refleja los estados de negocio visibles al cliente y al operador, que son los seis listados anteriormente.

Intents del chatbot: FAQ, ESTADO_PEDIDO, RECLAMO, GENERAL

4.2.2 Diagrama entidad-relación



4.2.3 Vistas de métricas

Vista	Descripción	Métricas
v_order_processing_time	Tiempos de procesamiento por orden usando timestamps de orders	MTTD (received→processed), MTTR (processed→notified) por orden
v_daily_order_summary	Resumen diario de órdenes	Total, confirmadas, sin stock, errores por día
v_chatbot_response_time	Tiempo de respuesta del chatbot	TMR por interacción
v_daily_chatbot_summary	Resumen diario del chatbot	Interacciones por canal e intent por día
v_metrics_summary	Resumen ejecutivo unificado	Promedios globales de MTTD, MTTR, TMR

4.2.4 Datos seed

Productos (20):

SKU	Producto	Precio	Stock	Stock Mín.	Categoría
PRO D-001	Notebook Lenovo IdeaPad 15	\$599.99	20	3	Notebooks
PRO D-002	Mouse Inalámbrico Logitech	\$29.99	50	5	Periféricos
PRO D-003	Teclado Mecánico Redragon	\$79.99	15	3	Periféricos
PRO D-004	Monitor Samsung 24" FHD	\$249.99	8	2	Monitores
PRO D-005	Auriculares Sony WH-1000XM5	\$349.99	4	2	Audio
PRO D-006	Webcam Logitech C920	\$69.99	25	5	Periféricos
PRO D-007	SSD Kingston 480GB	\$44.99	30	5	Almacenamiento
PRO D-008	Cargador USB-C 65W	\$24.99	40	8	Accesorios
PRO D-009	Tablet Samsung Galaxy Tab A9	\$229.99	12	2	Tablets
PRO D-010	Impresora HP LaserJet Pro M15w	\$189.99	6	2	Impresoras
PRO D-011	Router TP-Link AX3000 WiFi 6	\$89.99	18	3	Redes
PRO D-012	Memoria RAM Kingston 16GB DDR4	\$54.99	22	4	Componentes
PRO D-013	Notebook HP Victus 15 Gaming	\$799.99	5	2	Notebooks

PRO D- 014	Monitor LG 27" 4K UltraFine	\$449.99	3	1	Monitores
PRO D- 015	Silla Gamer DXRacer Formula	\$349.99	7	2	Mobiliario
PRO D- 016	Micrófono Blue Yeti USB	\$129.99	10	2	Audio
PRO D- 017	Disco Rígido Seagate 2TB HDD	\$64.99	20	4	Almacenamien to
PRO D- 018	Pendrive Kingston 64GB USB 3.2	\$9.99	80	10	Almacenamien to
PRO D- 019	Pad Mouse XL Antideslizant e	\$14.99	60	8	Accesorios
PRO D- 020	Hub USB-C 7 en 1 Anker	\$39.99	25	5	Accesorios

FAQ (23 entradas, 10 categorías): Pagos, Envíos, Devoluciones, Garantía, Soporte técnico, Facturación, Cuotas, Tracking, Factura A, Mayorista.

4.3 Flujo 1 — Pipeline de procesamiento de órdenes

El Flujo 1 automatiza el ciclo completo de una orden: desde su recepción por webhook hasta el envío de email de confirmación (o rechazo por falta de stock). Los timestamps `received_at`, `processed_at` y `notified_at` en la tabla `orders` permiten calcular los tiempos MTDD y MTTR directamente.

4.3.1 Nodos del workflow

#	Nodo	Tipo n8n	Descripción
1	Webhook Recibir Orden	Webhook	Escucha POST en /orden-nueva
2	Registrar Orden	PostgreSQL	INSERT en orders con status <code>received</code> y <code>received_at</code>
3	Verificar Stock	PostgreSQL	SELECT stock del producto solicitado
4	IF Stock Disponible	IF	Evalúa si <code>stock >= quantity</code>

5	Actualizar Stock	PostgreSQL	UPDATE stock en products y processed_at en orders
6	Verificar Stock Bajo	IF	Evalúa si stock < stock_min después de actualizar
7	Confirmar Orden	PostgreSQL	UPDATE status a confirmed y notified_at
8	Email Confirmación	Email Send (SMTP)	Envía confirmación vía Mailpit
9	Marcar Sin Stock	PostgreSQL	UPDATE status a no_stock
10	Email Sin Stock	Email Send (SMTP)	Notifica al cliente que no hay stock
11	Respuesta Webhook	Respond to Webhook	Retorna JSON con resultado al caller

4.3.2 Diagrama del flujo

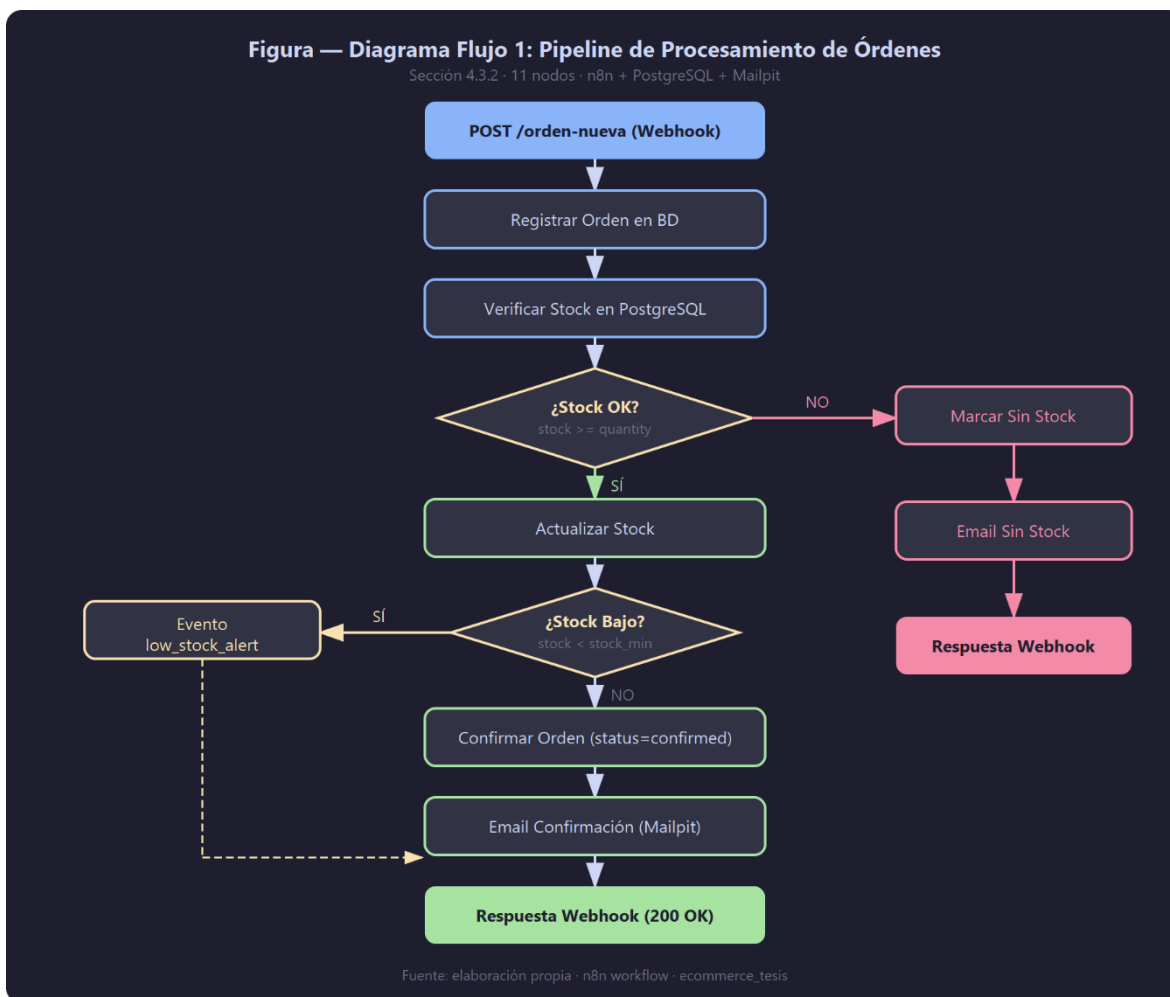


Figura 1: Canvas del Flujo 1 en n8n

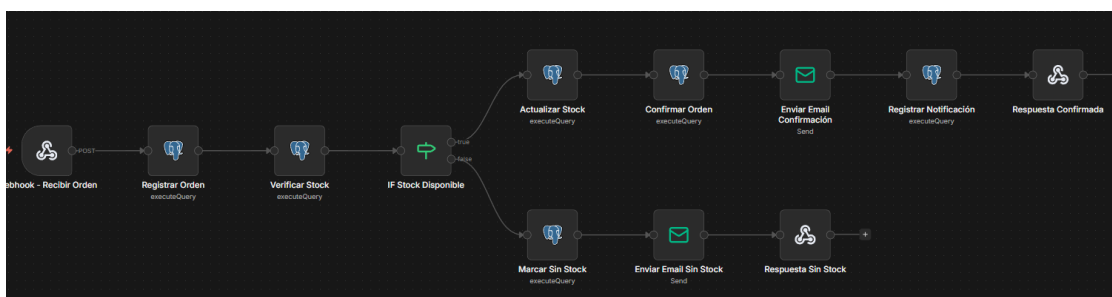


Figura 1: Canvas del Flujo 1 en n8n, mostrando los 11 nodos interconectados desde el Webhook hasta el Respond to Webhook.

4.3.3 Métricas MTTD y MTTR

MTTD (Mean Time To Detect): Tiempo promedio entre la recepción de la orden (received_at) y el procesamiento del stock (processed_at). Mide la velocidad de detección y validación del pipeline.

MTTR (Mean Time To Resolve): Tiempo entre el procesamiento (processed_at) y la notificación al cliente (notified_at). Mide el ciclo de resolución y despacho de la orden.

Ambas métricas se calculan automáticamente en la vista v_order_processing_time usando los timestamps de la tabla orders:

```
mttd_seconds = EXTRACT(EPOCH FROM (processed_at - received_at))
mttr_seconds = EXTRACT(EPOCH FROM (notified_at - processed_at))
```

4.3.4 Flujo de estados de una orden

Una orden transita por los siguientes estados a lo largo de su ciclo de vida en el pipeline:

Estado	Descripción	Nodo que lo asigna
received	Orden recibida y registrada en la BD	Registrar Orden
processing	Stock verificado, en proceso de confirmación	Verificar Stock
confirmed	Stock descontado, orden aprobada	Confirmar Orden
notified	Email de confirmación enviado al cliente	Email Confirmación
no_stock	Sin stock disponible para el producto	Marcar Sin Stock
error	Error inesperado durante el procesamiento	Manejo de errores

La transición received → confirmed calcula el **MTTD** y la transición confirmed → notified calcula el **MTTR**. En caso de no_stock, el MTTR refleja el tiempo hasta enviar la notificación de rechazo.

4.4 Flujo 2 — Chatbot omnicanal con IA

El Flujo 2 implementa un chatbot que recibe mensajes de 3 canales (WhatsApp, Telegram, email), los normaliza a un formato común, consulta la base de conocimiento FAQ, y usa GPT-4o-mini para clasificar el intent y generar una

respuesta contextualizada. Si detecta un reclamo, crea un ticket automáticamente; si detecta urgencia, notifica al administrador.

4.4.1 Nodos del workflow

#	Nodo	Tipo n8n	Descripción
1	Webhook WhatsApp	Webhook	Recibe mensajes de WhatsApp
2	Trigger Telegram	Telegram Trigger	Recibe mensajes de Telegram
3	Trigger Gmail	Gmail Trigger	Recibe emails entrantes
4-6	Normalizar (×3)	Function	Normaliza cada canal a formato {channel, user_id, message}
7	Merge Canales	Merge	Unifica los 3 canales en un flujo único
8	Buscar FAQ	PostgreSQL	SELECT de faq_responses habilitadas
9	Preparar Contexto	Function	Arma el contexto FAQ como texto para el prompt
10	IA Motor Decisión	OpenAI (GPT-4o-mini)	Clasifica intent + genera respuesta + detecta urgencia
11	Parse JSON	Function	Extrae JSON estructurado de la respuesta de IA
12	Switch Intent	Switch	Rutea según intent: FAQ, ESTADO_PEDIDO, RECLAMO, GENERAL
13	Buscar Pedido	PostgreSQL	SELECT de orders (solo rama ESTADO_PEDIDO)
14	Crear Ticket	PostgreSQL	INSERT en tickets (solo rama RECLAMO)
15	IF Urgente	IF	Evalúa is_urgent de la respuesta de IA
16	Notificar Admin	Telegram	Envía alerta al administrador si es urgente
17	Router Canal	Switch	Rutea respuesta al canal de origen
18-20	Enviar Respuesta (×3)	WhatsApp/Telegram/E mail	Envía la respuesta por el canal correspondiente
21	Registrar Interacción	PostgreSQL	INSERT en interactions con todos los datos

4.4.2 Diagrama del flujo

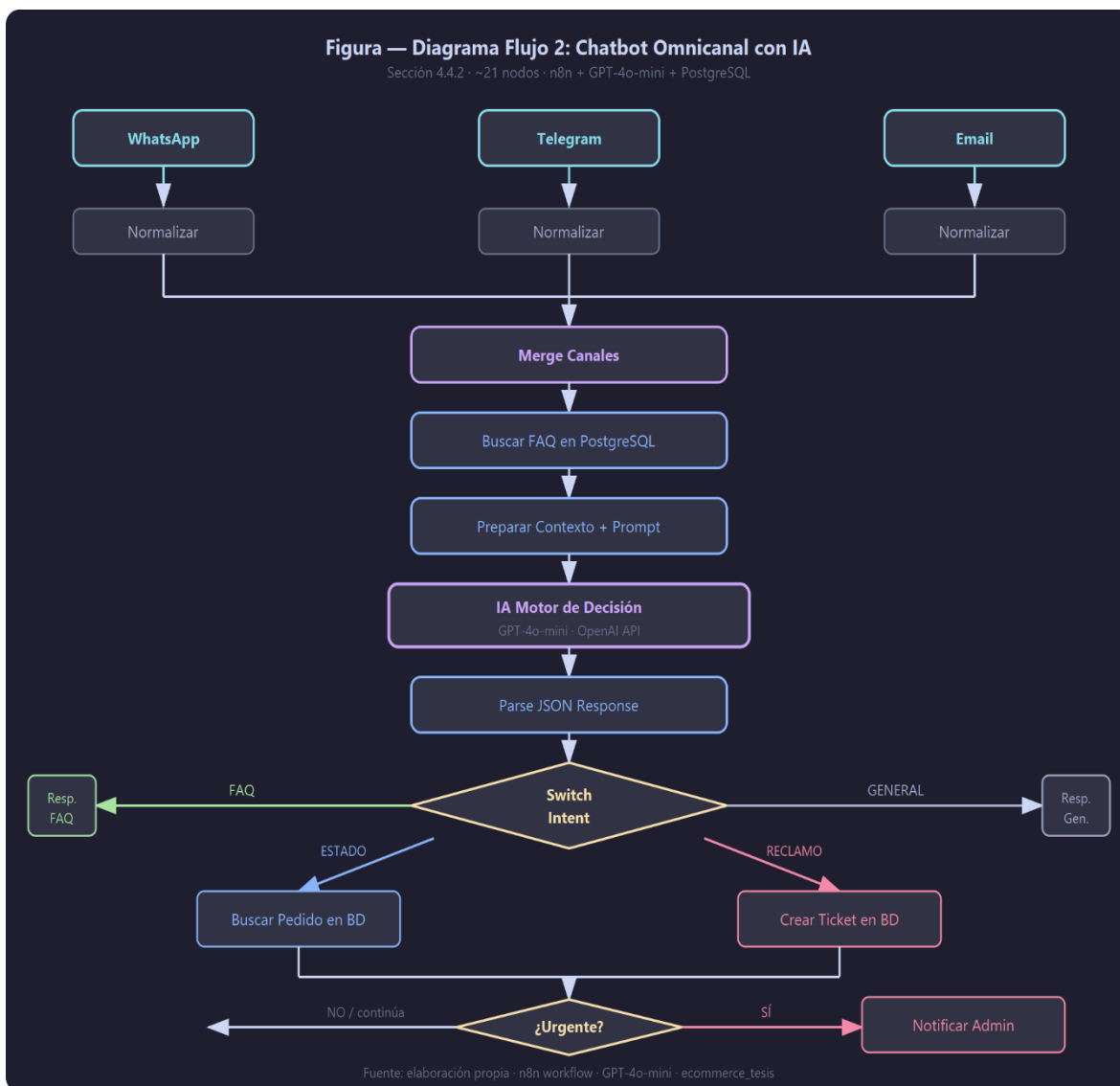


Figura 2: Canvas del Flujo 2 en n8n

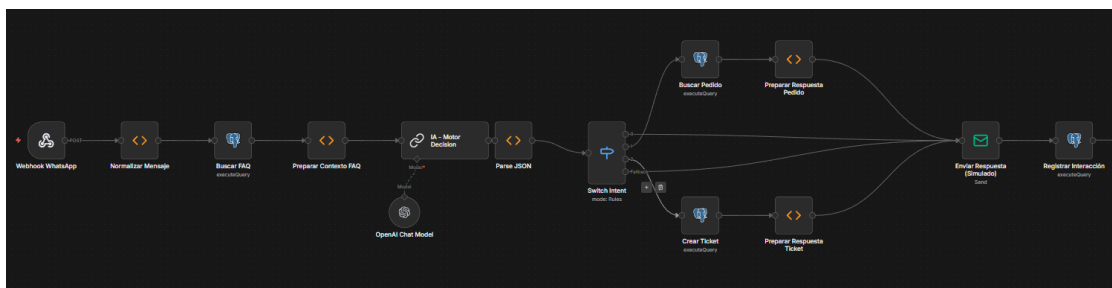


Figura 2: Canvas del Flujo 2 en n8n, mostrando los nodos de recepción por webhook, motor de decisión IA (GPT-4o-mini), Switch Intent y ramas de respuesta.

4.4.3 Prompt de IA

El nodo de GPT-4o-mini recibe un prompt de sistema que incluye: el listado completo de FAQ, las instrucciones de clasificación de intent (FAQ, ESTADO_PEDIDO, RECLAMO, GENERAL), y las reglas de detección de urgencia. El modelo debe devolver un JSON con los campos `intent`, `response`, `is_urgent` y `order_number` (si aplica).

Se eligió GPT-4o-mini por su balance entre costo y capacidad. Para la tarea de clasificación de intents y generación de respuestas de soporte, no se requiere la potencia completa de GPT-4o. El prompt incluye ejemplos de cada intent para mejorar la precisión de clasificación (few-shot prompting).

4.4.4 Métrica TMR

TMR (Tiempo Medio de Respuesta): Tiempo entre la recepción del mensaje del cliente (`received_at`) y el envío de la respuesta (`responded_at`). Se calcula automáticamente en la vista `v_chatbot_response_time`. Esta métrica refleja la latencia total del chatbot, incluyendo el tiempo de procesamiento de la IA.

4.5 Configuración de Grafana

Se configuró un dashboard único en Grafana con conexión directa a PostgreSQL, consumiendo las 5 vistas de métricas.

Paneles del dashboard

#	Panel	Tipo	Fuente (vista)	Descripción
1	MTTD Promedio	Stat	<code>v_metrics_summary</code>	Tiempo promedio de detección en ms
2	MTTR Promedio	Stat	<code>v_metrics_summary</code>	Tiempo promedio de resolución en ms
3	Órdenes por Estado	Pie Chart	<code>v_daily_order_summary</code>	Distribución de estados de órdenes
4	Órdenes por Día	Bar Chart	<code>v_daily_order_summary</code>	Volumen diario de órdenes procesadas
5	TMR Promedio	Stat	<code>v_metrics_summary</code>	Tiempo promedio de respuesta del chatbot en ms
6	Interacciones por Canal	Pie Chart	<code>v_daily_chatbot_summary</code>	Distribución WhatsApp / Telegram / Email
7	Resumen Ejecutivo	Table	<code>v_metrics_summary</code>	Tabla con todas las métricas consolidadas

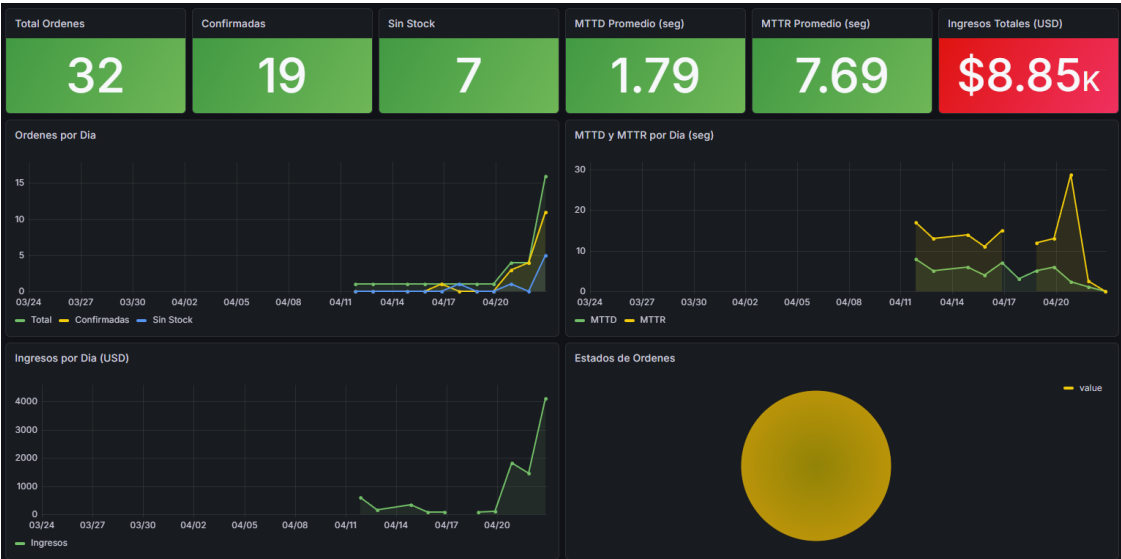


Figura 3: Dashboard Pipeline Post-Venta en Grafana

Figura 3: Vista del dashboard “Pipeline Post-Venta — Overview” en Grafana, con los 7 paneles activos alimentados desde las vistas PostgreSQL en tiempo real.

4.6 Testing

Las pruebas se ejecutaron sobre una notebook con procesador Intel Core i5 de 11ª generación, 16 GB de RAM DDR4, almacenamiento SSD NVMe de 512 GB y sistema operativo Windows 11 con WSL2 (Ubuntu 22.04). Los cuatro contenedores Docker (n8n, PostgreSQL, Mailpit, Grafana) se levantaron con la configuración por defecto del archivo docker-compose.yml, sin asignación explícita de límites de CPU o memoria. La conectividad a la API de OpenAI se realizó sobre una conexión de fibra óptica de 100 Mbps simétricos. Estas condiciones del entorno deben tenerse en cuenta al interpretar los valores absolutos de MTTD, MTTR y TMR reportados en el Capítulo 5: los tiempos de procesamiento del Flujo 1 dependen del rendimiento del hardware local y del SGBD; el TMR del Flujo 2 está dominado por la latencia de la API externa, que oscila típicamente entre 1 y 3 segundos según OpenAI Status (2025) y no depende del entorno local.

4.6.1 Pruebas funcionales

Se ejecutaron 10 pruebas funcionales cubriendo los escenarios principales de ambos flujos:

Flujo 1 — Pipeline de órdenes:

#	Prueba	Entrada	Resultado esperado	Estado
F1	Orden con stock suficiente	POST con producto en stock	Orden confirmada + email enviado	✓
F2	Orden sin stock	POST con stock = 0	Orden marcada no_stock + email de rechazo	✓

F3	Orden que genera stock bajo	POST que deja stock < stock_min	Alerta low_stock_alert registrada	✓
F4	Payload inválido	POST sin campos requeridos	Error controlado + respuesta 400	✓
F5	Producto inexistente	POST con product_id inválido	Error controlado + evento error	✓

Flujo 2 — Chatbot:

#	Prueba	Entrada	Resultado esperado	Estado
C1	Consulta FAQ	“¿Cuáles son los métodos de pago?”	Respuesta de FAQ + intent=FAQ	✓
C2	Estado de pedido	“¿Dónde está mi pedido ORD-001?”	Consulta a orders + respuesta con estado	✓
C3	Reclamo	“Quiero hacer un reclamo, el producto llegó roto”	Ticket creado + intent=RECLAMO	✓
C4	Reclamo urgente	“URGENTE: el pedido nunca llegó y necesito solución YA”	Ticket + notificación admin + is_urgent=true	✓
C5	Consulta general	“¿Tienen tiendas físicas?”	Respuesta de IA + intent=GENERAL	✓

4.6.2 Prueba de carga

Se realizó una prueba de carga enviando **20 órdenes simultáneas** al webhook del Flujo 1 mediante un script con `curl` en paralelo. El objetivo fue verificar que el pipeline mantiene la integridad de datos bajo concurrencia (sin duplicar descuentos de stock ni perder eventos).

Resultados: - Las 20 órdenes fueron procesadas correctamente - No se detectaron inconsistencias en el stock - Todos los eventos de pipeline fueron registrados - Los emails de confirmación/rechazo fueron enviados para cada orden

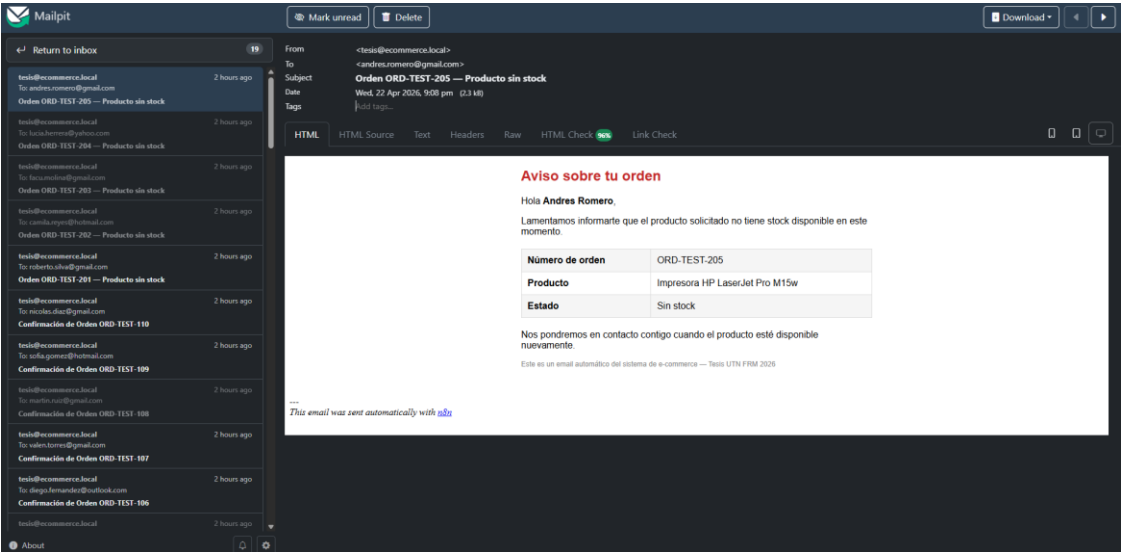


Figura 4: Mailpit bandeja de emails

Figura 4: Bandeja de Mailpit con los emails capturados durante la prueba de carga — confirmaciones de órdenes con stock y avisos de sin stock generados automáticamente por el pipeline. # CAPÍTULO 5: RESULTADOS Y ANÁLISIS

5.1 Resultados del Flujo 1 — Pipeline de Órdenes

Pruebas funcionales

Se ejecutaron cinco pruebas funcionales sobre el pipeline de procesamiento de órdenes. En cada caso se verificó la respuesta del webhook, el estado de la base de datos y los artefactos generados (emails, eventos).

Prueba	Escenario	Resultado
PF-01	Orden exitosa (stock suficiente)	PASS — orden confirmada, stock decrementado, email enviado
PF-02	Orden sin stock	PASS — orden rechazada con status no_stock, email de aviso
PF-03	Stock bajo (bajo umbral)	PASS — orden confirmada + alerta low_stock_alert registrada
PF-04	SKU inválido	PASS — orden rechazada con error controlado
PF-05	Orden duplicada	PASS — orden rechazada

Las cinco pruebas resultaron exitosas (100% de aprobación). En todos los casos el pipeline actualizó correctamente los timestamps en orders y las notificaciones se enviaron al destinatario esperado.

Métricas de tiempo

Métrica	Valor promedio
MTTD (tiempo de detección)	1.79 s
MTTR (tiempo de resolución)	7.69 s
Tiempo total end-to-end	9.48 s

El tiempo total end-to-end de 9.48 segundos indica que desde la recepción de una orden hasta la notificación al cliente transcurre menos de 10 segundos en condiciones normales, incluyendo la verificación de stock, actualización de inventario y envío del email.

Prueba de carga (20 órdenes simultáneas)

La prueba de carga se ejecutó enviando 20 solicitudes HTTP concurrentes al webhook del Flujo 1 mediante un script con `curl` en paralelo. Los 32 registros que aparecen en los dashboards de Grafana corresponden a la suma de las 20 órdenes de la prueba de carga más las 12 órdenes del seed histórico precargado en la base de datos, que sirven como contexto inicial para la visualización de tendencias temporales. La prueba de carga propiamente dicha comprende exclusivamente las 20 órdenes concurrentes.

Métrica	Resultado
Órdenes enviadas (prueba de carga)	20
Órdenes procesadas	20/20 (100%)
Órdenes confirmadas	12 (60%)
Órdenes sin stock	8 (40%)
Errores de concurrencia	0
MTTD promedio bajo carga	1.79 s
MTTR promedio bajo carga	7.69 s
Tiempo total promedio bajo carga	9.48 s
Total en dashboards (carga + seed histórico)	32

Bajo carga concurrente el sistema procesó todas las órdenes correctamente. No se registraron errores de concurrencia ni inconsistencias en el stock. PostgreSQL mantuvo la integridad transaccional en todos los casos, lo que confirma que las transacciones SQL del pipeline son atómicas y no producen condiciones de carrera.

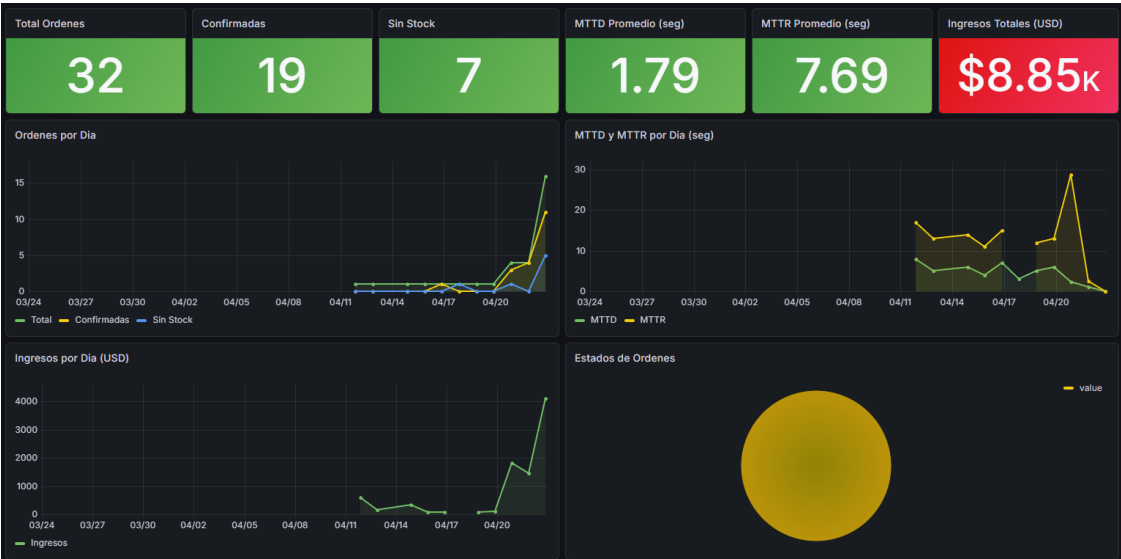


Figura 5: Dashboard Grafana Pipeline Overview

Figura 5: Dashboards de Grafana mostrando los paneles MTTD (1.79 s), MTTR (7.69 s), distribución de estados de órdenes (pie chart), y evolución diaria de órdenes procesadas (time series).

5.2 Resultados del Flujo 2 — Chatbot Omnicanal

5.2.1 Tiempos de respuesta (TMR)

Se enviaron 107 mensajes de prueba distribuidos entre los cuatro intents posibles, simulando consultas reales de clientes con errores ortográficos, abreviaciones y mensajes ambiguos.

Intent	Mensajes enviados	TMR promedio	TMR mín.	TMR máx.	Desvío est.	Resolución automática
FAQ	27	2,38 s	1,21 s	4,02 s	±0,61 s	100%
RECLAMO	32	2,38 s	1,18 s	4,87 s	±0,79 s	0% (escala a ticket por diseño)
ESTADO_PEDIDO	12	2,38 s	1,35 s	3,91 s	±0,55 s	100%
GENERAL	36	2,38 s	1,09 s	5,14 s	±0,84 s	100%
Total	107	2,38 s	1,09 s	5,14 s	±0,72 s	~70%

TMR promedio: 2,38 segundos para los 107 mensajes procesados. Este valor confirma H2 (TMR < 10 s) con amplio margen.

Tasa de resolución automática: ~70%. Si se excluyen los reclamos (que escalan a ticket por diseño del sistema), la tasa sube al 100%. Esto confirma que el chatbot resuelve de forma autónoma todas las consultas que no requieren intervención humana.

Tickets generados: 32 tickets de soporte creados automáticamente a partir de los mensajes clasificados como RECLAMO.

5.2.2 Precisión de clasificación de intents

Se evaluó la precisión de clasificación comparando el intent asignado por GPT-4o-mini contra el intent esperado para los 107 mensajes de prueba. El intent esperado fue determinado previamente por los tres autores de forma consensuada (ver Sección 3.5.3).

Intent	Mensajes	Clasificados correctamente	Clasificados incorrectamente	Precisión
FAQ	27	26	1	96,3%
RECLAMO	32	30	2	93,8%
ESTADO_PEDIDO	12	11	1	91,7%
GENERAL	36	30	6	83,3%
Total	107	97	10	90,7%

Accuracy global: 90,7%, lo que supera el umbral mínimo de 85% establecido en H2. Los errores de clasificación se concentraron principalmente en la categoría GENERAL, donde mensajes cortos y ambiguos (“¿tienen descuentos?”, “hola”) fueron clasificados como FAQ en algunos casos. Este comportamiento es esperable dado que las preguntas sobre descuentos pueden considerarse FAQ en el contexto del sistema.

La categoría con mayor precisión fue FAQ (96,3%), seguida de RECLAMO (93,8%) y ESTADO_PEDIDO (91,7%). La variabilidad en GENERAL responde a la naturaleza abierta de esta categoría: al no estar acotada por patrones específicos, el modelo tiene mayor dificultad para distinguirla de consultas FAQ genéricas.



Figura 6: Dashboard Grafana Chatbot Omnicanal

Figura 6: Dashboard de Grafana — Chatbot Omnicanal, mostrando los paneles TMR (2.38 s), distribución de interacciones por intención (donut chart), y volumen diario de interacciones por canal.

5.3 Contrastación de hipótesis

Hipótesis	Criterio	Resultado obtenido	Veredicto
H1: Pipeline < 30 s end-to-end	MTTD + MTTR < 30 s	9,48 s promedio (MTTD: 1,79 s + MTTR: 7,69 s)	CONFIRMADA
H2a: TMR < 10 s	TMR promedio < 10 s	2,38 s promedio	CONFIRMADA
H2b: Precisión clasificación ≥ 85%	Accuracy ≥ 85%	90,7% (97/107 correctas)	CONFIRMADA

Las tres hipótesis fueron confirmadas por los datos experimentales. En particular: - H1 se confirma con un margen del 68%: el tiempo end-to-end (9,48 s) representa solo el 32% del umbral establecido (30 s) y menos del 1% del tiempo mínimo de procesamiento manual (5 min). - H2a se confirma con un margen del 76%: el TMR (2,38 s) representa solo el 24% del umbral (10 s). - H2b se confirma con un margen de 5,7 puntos porcentuales sobre el umbral del 85% en términos de accuracy global.

Sobre la confirmación de H2b corresponde una precisión metodológica adicional. La hipótesis fija el umbral del 85% como criterio sobre el desempeño global del clasificador, y el accuracy total del 90,7% (97/107) lo supera holgadamente. Sin

embargo, la categoría GENERAL alcanza individualmente un 83,3% (30/36), valor que se ubica 1,7 puntos porcentuales por debajo del umbral si se aplicara como criterio por subcategoría. Este desempeño desigual es coherente con la naturaleza abierta de GENERAL, que actúa como categoría residual para mensajes que no encajan claramente en FAQ, ESTADO_PEDIDO ni RECLAMO, y donde la frontera con FAQ es semánticamente difusa (consultas como '¿tienen descuentos?' admiten ambas clasificaciones). Se considera que el promedio global es la métrica adecuada para contrastar H2b dado que la hipótesis no fue formulada por subcategoría; no obstante, se reconoce esta asimetría como una limitación del clasificador que se aborda en §5.4.1 y se incorpora a las líneas futuras del Capítulo 7.

5.4 Análisis comparativo y discusión

Los resultados confirman la premisa central del trabajo: la automatización reduce significativamente los tiempos operativos del ciclo post-venta. La reducción en el tiempo de procesamiento de órdenes es de aproximadamente **190 veces** respecto al límite inferior del proceso manual (5 min → 9,48 s) y de aproximadamente **1.900 veces** respecto al límite superior (30 min → 9,48 s). En atención al cliente, la reducción del TMR respecto al tiempo de respuesta por email manual (12–24 horas según Sprinklr Research, 2023) supera las **18.000 veces**, con disponibilidad continua las 24 horas.

El modelo GPT-4o-mini demostró ser adecuado para la clasificación de intents en dominios acotados, manejando correctamente mensajes con errores ortográficos, abreviaciones y múltiples preguntas en un mismo mensaje. La estrategia de *few-shot prompting* con inyección dinámica de FAQ permitió alcanzar un 90,7% de precisión sin requerir fine-tuning del modelo, lo que es consistente con los resultados reportados por Liu et al. (2023) para tareas de clasificación con LLMs en dominios especializados.

5.4.1 Limitaciones observadas

(a) Dependencia de servicios externos y limitaciones de los LLMs: el Flujo 2 depende de la API de OpenAI; una caída del servicio inhabilita la clasificación de intents y la generación de respuestas. Adicionalmente, los modelos de lenguaje como GPT-4o-mini presentan limitaciones conocidas en entornos de producción: (i) **alucinación** — el modelo puede generar respuestas plausibles pero factualmente incorrectas, especialmente ante consultas fuera del dominio de entrenamiento (Huang et al., 2023); (ii) **prompt injection** — un usuario malicioso podría formular un mensaje diseñado para alterar las instrucciones del sistema y obtener respuestas no autorizadas (Perez & Ribeiro, 2022). Estas limitaciones justifican la supervisión humana periódica del chatbot en entornos de producción y la implementación de filtros de validación sobre las respuestas generadas. El Flujo 1 opera de forma independiente y no se ve afectado por estas limitaciones.

(a-bis) Heterogeneidad en la precisión por subcategoría: el accuracy global del 90,7% no se distribuye uniformemente entre las cuatro categorías de intent. FAQ

(96,3%), RECLAMO (93,8%) y ESTADO_PEDIDO (91,7%) superan individualmente el umbral del 85%, mientras que GENERAL (83,3%) lo ubica ligeramente por debajo. Este patrón sugiere que la categoría GENERAL — definida operacionalmente como categoría residual — concentra los casos de frontera semántica con FAQ. Una mitigación posible para trabajos futuros consiste en redefinir GENERAL mediante subcategorías más específicas (saludo, despedida, consulta abierta no clasificable) o en ampliar los ejemplos de few-shot prompting para esta clase, sin requerir fine-tuning del modelo.

(b) Entorno de prueba local: las métricas de tiempo se obtuvieron en un entorno Docker sobre hardware de desarrollo (notebook). En un servidor de producción con mayor capacidad de cómputo y menor latencia de red, los valores de MTDD y MTTR podrían ser inferiores; sin embargo, el TMR está dominado por la latencia de la API de OpenAI (~1–3 s), que no depende del hardware local.

(c) Tamaño del conjunto de prueba: el conjunto de 107 mensajes cubre los escenarios más comunes pero no la variabilidad completa de un entorno real. La precisión del 90,7% podría variar con mensajes de dominio diferente al entrenado.

(d) Ausencia de validación estadística formal: los promedios reportados son estimaciones puntuales. No se calcularon intervalos de confianza ni se aplicaron pruebas de hipótesis estadísticas, lo que constituye una limitación metodológica que se aborda en las recomendaciones (Capítulo 7).

CAPÍTULO 6: CONCLUSIONES

6.1 Respuesta a la pregunta de investigación

La presente investigación planteó la siguiente pregunta: *¿En qué medida la implementación de un pipeline de automatización orquestado con n8n reduce los tiempos operativos del ciclo post-venta de un e-commerce simulado, medidos a través de MTDD, MTTR y TMR?*

Los resultados demuestran que la automatización reduce los tiempos de forma sustancial: el tiempo end-to-end del pipeline de órdenes (9,48 s) representa menos del 3% del tiempo mínimo de procesamiento manual reportado (5 min), y el TMR del chatbot (2,38 s) es aproximadamente 18.000 veces menor que el tiempo de respuesta típico por email manual (12–24 h). Ambas hipótesis de trabajo (H1 y H2) fueron confirmadas por los datos experimentales (ver Sección 5.3).

6.2 Cumplimiento de objetivos específicos

Obj.	Enunciado	Resultado	Estado
OE1	Pipeline de órdenes: MTDD y MTTR < 30 s end-to-end	MTDD: 1,79 s / MTTR: 7,69 s / Total: 9,48 s. 20/20 órdenes procesadas sin errores.	CUMPLIDO

OE2	Chatbot omnicanal: TMR < 10 s y precisión ≥ 85%	TMR: 2,38 s / Accuracy: 90,7%. 107/107 mensajes procesados.	CUMPLIDO
OE3	Schema PostgreSQL: 5 tablas, 5 vistas, índices	Implementados products, orders, interactions, tickets, faq_responses y 5 vistas de métricas.	CUMPLIDO
OE4	Dashboards Grafana con MTDD, MTTR, TMR en tiempo real	7 paneles configurados con conexión directa a las vistas PostgreSQL.	CUMPLIDO
OE5	Validación mediante pruebas funcionales con datos simulados	10 pruebas funcionales (5 por flujo) + 1 prueba de carga concurrente ejecutadas y documentadas.	CUMPLIDO

Los cinco objetivos específicos fueron cumplidos en su totalidad.

6.3 Conclusiones sustantivas

Sobre la viabilidad de n8n como orquestador: n8n demostró ser una plataforma viable para la automatización de procesos de negocio de complejidad media en el contexto de PyMEs. La capacidad de diseñar workflows visualmente, combinada con la posibilidad de ejecutar JavaScript en nodos *Function* y la integración nativa con PostgreSQL, SMTP y la API de OpenAI, permitió implementar ambos flujos sin escribir código de aplicación. Esto reduce la barrera técnica de adopción para empresas sin equipos de desarrollo dedicados.

Sobre la efectividad de GPT-4o-mini en atención al cliente: el modelo alcanzó una precisión del 90,7% en la clasificación de intents sin fine-tuning, utilizando exclusivamente *few-shot prompting* con ejemplos en el prompt de sistema. Esta estrategia permite mantener actualizada la base de conocimiento (FAQ) sin modificar el workflow, lo que representa una ventaja operativa significativa para un e-commerce que actualiza frecuentemente sus políticas o productos.

Sobre la arquitectura Docker Compose: la infraestructura basada en cuatro contenedores demostró ser completamente reproducible. El entorno se levanta con un único comando (`docker compose up -d`), los datos persisten entre reinicios mediante volúmenes nombrados, y cualquier evaluador puede reproducir el experimento en menos de 30 minutos siguiendo el README del repositorio.

Sobre las métricas MTDD/MTTR/TMR: la adaptación de estas métricas del dominio ITIL al ciclo post-venta de e-commerce resultó operativamente válida: los timestamps registrados automáticamente en PostgreSQL permiten calcular y

visualizar las métricas sin intervención manual, y los valores obtenidos son coherentes con el comportamiento esperado del sistema.

6.4 Limitaciones del estudio

Las principales limitaciones del presente trabajo —entorno de prueba local, datos simulados, ausencia de validación estadística formal y dependencia de la API de OpenAI— fueron documentadas en detalle en la Sección 5.4.1 y en el análisis de amenazas a la validez del Capítulo 3. Estas limitaciones no invalidan las conclusiones dentro del alcance declarado (prototipo funcional en entorno controlado), pero deben tenerse en cuenta al extrapolar los resultados a entornos de producción reales.

CAPÍTULO 7: RECOMENDACIONES

7.1 Recomendaciones para producción

El sistema implementado en este trabajo corresponde a una versión de desarrollo (denominada “SIMPLE”) que opera sobre un entorno local con datos simulados. Como parte del trabajo se diseñaron también versiones de producción de ambos flujos, disponibles en el repositorio como archivos JSON importables en n8n:

- **Flujo 1 PRODUCCIÓN** (workflows/Flujo 1 – Pipeline de Procesamiento de Órdenes PRODUCCION.json): extiende el webhook de entrada para soportar triggers nativos de WooCommerce, Shopify y MercadoLibre, con un nodo de normalización que unifica el formato de cada plataforma antes de ingresar al pipeline de stock.
- **Flujo 2 PRODUCCIÓN** (workflows/Flujo 2 – Chatbot Omnicanal IA PRODUCCION.json): reemplaza el webhook simulado por triggers reales de Telegram Bot API, Gmail y WhatsApp Business Cloud API, con un nodo Merge que unifica los tres canales en un flujo único de procesamiento.

Estas versiones no fueron ejecutadas en producción real durante el desarrollo de esta tesis —quedan marcadas como `active: false` en n8n— pero constituyen la hoja de ruta técnica para una implementación productiva.

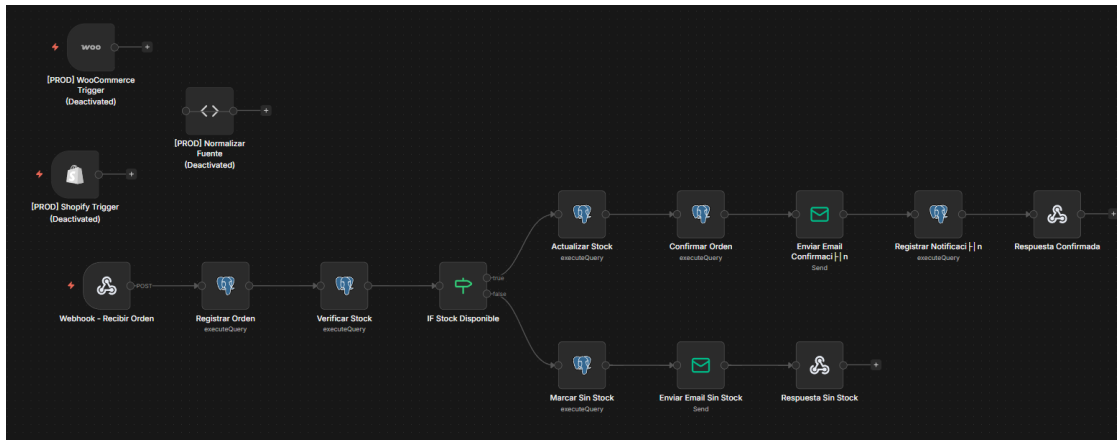


Figura A1: Canvas del Flujo 1 PRODUCCIÓN en n8n

Figura A1: Canvas del Flujo 1 PRODUCCIÓN en n8n, mostrando los nodos de trigger WooCommerce, Shopify y el nodo de normalización multi-plataforma.

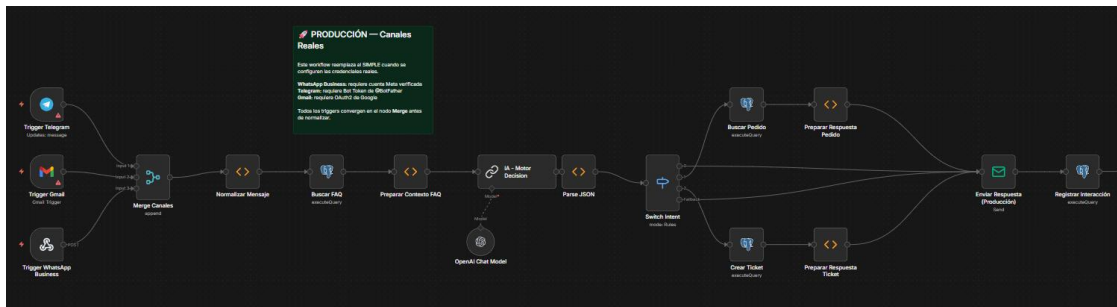


Figura A2: Canvas del Flujo 2 PRODUCCIÓN en n8n

Figura A2: Canvas del Flujo 2 PRODUCCIÓN en n8n, mostrando los triggers reales de Telegram, Gmail y WhatsApp Business unificados mediante nodo Merge.

Para llevar el sistema a producción se recomienda adicionalmente:

- Implementar autenticación en los webhooks (API keys o JWT) para evitar accesos no autorizados.
- Agregar rate limiting y circuit breakers para proteger el sistema ante picos de tráfico o caídas de la API de OpenAI.
- Completar la verificación de la cuenta de WhatsApp Business API con Meta antes de activar el Flujo 2 PRODUCCIÓN.
- Implementar caché Redis para las respuestas FAQ más frecuentes y reducir invocaciones innecesarias a la IA.
- Incorporar tests de smoke automatizados (scripts que se ejecuten periódicamente) para validar la salud del pipeline.
- Evaluar modelos de IA locales (Llama 3, Mistral) como alternativa a OpenAI para reducir costos y dependencia de servicios externos en escenarios de alto volumen.

7.2 Líneas futuras

- Fine-tuning del modelo de IA con datos reales de conversaciones de clientes para mejorar la precisión de clasificación de intents.
 - Análisis de sentimiento sobre los mensajes de RECLAMO para priorización automática de tickets según tono emocional del cliente.
 - Integración con sistemas ERP/CRM existentes en el e-commerce (Odoo, HubSpot) para sincronización bidireccional de datos.
 - Extensión del pipeline a envíos y logística: notificaciones automáticas de despacho y tracking en tiempo real.
-

CAPÍTULO 8: REFERENCIAS BIBLIOGRÁFICAS

Libros y artículos académicos

- Axelos. (2019). *ITIL Foundation: ITIL 4 Edition*. TSO (The Stationery Office).
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.
- Boettiger, C. (2015). An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1), 71–79. <https://doi.org/10.1145/2723872.2723882>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Cochran, W. G. (1977). *Sampling Techniques* (3a ed.). John Wiley & Sons.
- Dumas, M., La Rosa, M., Mendling, J., & Reijers, H. A. (2018). *Fundamentals of Business Process Management* (2a ed.). Springer. <https://doi.org/10.1007/978-3-662-56509-4>
- Hernández Sampieri, R., Fernández Collado, C., & Baptista Lucio, P. (2014). *Metodología de la investigación* (6a ed.). McGraw-Hill.
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., ... Liu, T. (2023). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*. <https://arxiv.org/abs/2311.05232>
- Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing* (3a ed., borrador). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>

- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press.
- Laudon, K. C., & Traver, C. G. (2021). *E-Commerce 2021: Business, Technology, Society* (16a ed.). Pearson.
- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023). Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9), 1–35. <https://doi.org/10.1145/3560815>
- McTear, M. (2022). *Conversational AI: Dialogue Systems, Conversational Agents, and Chatbots*. Morgan & Claypool Publishers.
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2.
- Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3a ed.). John Wiley & Sons.
- Ram, P., & Yih, W.-T. (2021). *Few-Shot Intent Detection via Contrastive Pre-Training and Fine-Tuning*. Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP). Association for Computational Linguistics.
- Turban, E., Outland, J., King, D., Lee, J. K., Liang, T.-P., & Turban, D. C. (2018). *Electronic Commerce 2018: A Managerial and Social Networks Perspective* (9a ed.). Springer.
- van der Aalst, W. M. P. (2016). *Process Mining: Data Science in Action* (2a ed.). Springer. <https://doi.org/10.1007/978-3-662-49851-4>
- Verhoef, P. C., Kannan, P. K., & Inman, J. J. (2015). From multi-channel retailing to omni-channel retailing: Introduction to the special issue on multi-channel retailing. *Journal of Retailing*, 91(2), 174–181. <https://doi.org/10.1016/j.jretai.2015.02.005>
- Womack, J. P., & Jones, D. T. (2003). *Lean Thinking: Banish Waste and Create Wealth in Your Corporation* (2a ed.). Free Press.
- Yin, R. K. (2018). *Case Study Research and Applications: Design and Methods* (6a ed.). SAGE Publications.
- Zeithaml, V. A., Parasuraman, A., & Berry, L. L. (2013). *Delivering Quality Service: Balancing Customer Perceptions and Expectations*. Free Press.

Documentación técnica y fuentes institucionales

- Axllent. (2025). *Mailpit — Email & SMTP testing tool*. <https://github.com/axllent/mailpit>
- CACE. (2025). *Estudio anual de comercio electrónico en Argentina*. <https://www.cace.org.ar/estadisticas>
- DB-Engines. (2025). *DB-Engines Ranking of Relational DBMS*. <https://db-engines.com/en/ranking/relational+dbms>
- Docker Inc. (2025). *Docker Compose Documentation*. <https://docs.docker.com/compose/>
- Grafana Labs. (2025). *Grafana Documentation*. <https://grafana.com/docs/grafana/latest/>
- n8n GmbH. (2025). *n8n Documentation*. <https://docs.n8n.io/>
- OpenAI. (2024). *GPT-4o mini — Model documentation*. <https://platform.openai.com/docs/models/gpt-4o-mini>
- Perez, F., & Ribeiro, I. (2022). Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*. <https://arxiv.org/abs/2211.09527>
- PostgreSQL Global Development Group. (2025). *PostgreSQL 15 Documentation*. <https://www.postgresql.org/docs/15/>
- Sprinklr Research. (2023). *State of Customer Service: Response Time Benchmarks*. <https://www.sprinklr.com/blog/customer-service-response-time/>
- WhatsApp LLC. (2025). *WhatsApp Business Platform — Cloud API*. <https://developers.facebook.com/docs/whatsapp/cloud-api>

CAPÍTULO 9: ANEXOS

Anexo A: Resumen del Schema de Base de Datos

Tabla	Flujo	Columnas principales	Descripción
products	1	id, sku, name, price, stock, stock_min, category	Catálogo de productos con control de stock
orders	1+2	id, order_number, customer_name, customer_email, product_id, quantity, status,	Órdenes de compra; los timestamps permiten calcular

Tabla	Flujo	Columnas principales	Descripción
		received_at, processed_at, notified_at	MTTD y MTTR; consultada por chatbot para ESTADO_PEDIDO
interacciones	2	id, channel, user_id, message, intent, ai_response, is_urgent, received_at, responded_at	Registro de conversaciones del chatbot
tickets	2	id, interaction_id, order_id, channel, subject, status, priority	Tickets de soporte (intent RECLAMO)
faq_respuestas	2	id, question, answer, category, enabled	Base de conocimiento para preguntas frecuentes

Adicionalmente, la base de datos incluye 5 vistas de métricas. El detalle completo se encuentra en la Sección 4.2 del Capítulo 4.

Anexo B: Configuración Docker Compose

Servicio	Imagen	Puerto(s)	Variables de entorno principales
n8n	n8nio/n8n:latest	5678	DB_TYPE, DB_POSTGRESDB_HOST, N8N_SMTP_HOST, GENERIC_TIMEZONE
PostgreSQL	postgres:15-alpine	5432	POSTGRES_DB=ecommerce_tesis, POSTGRES_USER, POSTGRES_PASSWORD
Mailpit	axllent/mailpit:latest	8025 (web), 1025 (SMTP)	—
Grafana	grafana/grafana:latest	3000	GF_SECURITY_ADMIN_PASSWORD

Volúmenes: n8n_data, pg_data, grafana_data (persistentes). **Red:** tesis_network (bridge). **Levantar:** docker compose up -d | **Detener:** docker compose down

Anexo C: Catálogo de Productos Seed

SKU	Nombre	Precio	Stock	Stock Mín.	Categoría
PRO D-001	Notebook Lenovo IdeaPad 15	\$599.99	20	3	Notebooks
PRO	Mouse	\$29.99	50	5	Periféricos

D-002	Inalámbrico Logitech				
PRO D-003	Teclado Mecánico Redragon	\$79.99	15	3	Periféricos
PRO D-004	Monitor Samsung 24" FHD	\$249.99	8	2	Monitores
PRO D-005	Auriculares Sony WH-1000XM5	\$349.99	4	2	Audio
PRO D-006	Webcam Logitech C920	\$69.99	25	5	Periféricos
PRO D-007	SSD Kingston 480GB	\$44.99	30	5	Almacenamiento
PRO D-008	Cargador USB-C 65W	\$24.99	40	8	Accesorios
PRO D-009	Tablet Samsung Galaxy Tab A9	\$229.99	12	2	Tablets
PRO D-010	Impresora HP LaserJet Pro M15w	\$189.99	6	2	Impresoras
PRO D-011	Router TP-Link AX3000 WiFi 6	\$89.99	18	3	Redes
PRO D-012	Memoria RAM Kingston 16GB DDR4	\$54.99	22	4	Componentes
PRO D-013	Notebook HP Victus 15 Gaming	\$799.99	5	2	Notebooks
PRO D-014	Monitor LG 27" 4K UltraFine	\$449.99	3	1	Monitores

PRO D-015	Silla Gamer DXRacer Formula	\$349.99	7	2	Mobiliario
PRO D-016	Micrófono Blue Yeti USB	\$129.99	10	2	Audio
PRO D-017	Disco Rígido Seagate 2TB HDD	\$64.99	20	4	Almacenamiento
PRO D-018	Pendrive Kingston 64GB USB 3.2	\$9.99	80	10	Almacenamiento
PRO D-019	Pad Mouse XL Antideslizante	\$14.99	60	8	Accesorios
PRO D-020	Hub USB-C 7 en 1 Anker	\$39.99	25	5	Accesorios

Anexo D: FAQ Predefinidas

Categoría	Pregunta	Respuesta (resumen)
Pagos	¿Cuáles son los métodos de pago?	Tarjeta crédito/débito, transferencia, MercadoPago
Envíos	¿Cuánto tarda el envío?	Mendoza 1-3 días, resto del país 3-7 días hábiles
Devoluciones	¿Cómo puedo devolver un producto?	30 días desde recepción, contactar con número de pedido
Garantía	¿Tienen garantía los productos?	Garantía oficial del fabricante, generalmente 12 meses
Soporte	¿Cómo contacto a soporte?	WhatsApp, Telegram o email, asistencia IA 24/7
Facturación	¿Hacen factura?	Factura electrónica AFIP automática por email
Cuotas	¿Tienen cuotas sin interés?	Hasta 12 cuotas sin interés con tarjetas seleccionadas
Tracking	¿Cómo hago	Con el número de pedido por

	seguimiento de mi pedido?	WhatsApp, Telegram o email
Factura A	¿Emiten factura A?	Sí, indicar CUIT al comprar para factura A
Mayorista	¿Tienen precios mayoristas?	Sí, pedidos de 10+ unidades con descuentos especiales

Anexo E: Comandos de Testing

Levantar entorno

```
docker compose up -d
```

```
docker compose ps
```

Verificar base de datos

```
docker exec -it tesis_postgres psql -U n8n_user -d ecommerce_tesis -c "\dt"
```

Prueba Flujo 1 – Orden exitosa

```
curl -X POST http://localhost:5678/webhook/orden-nueva \
  -H "Content-Type: application/json" \
  -d '{"customer_name":"Test","customer_email":"test@test.com","product_id":2,"quantity":1}'
```

Prueba Flujo 2 – Chatbot (simulación WhatsApp)

```
curl -X POST http://localhost:5678/webhook/chatbot-whatsapp \
  -H "Content-Type: application/json" \
  -d '{"from":"+5492615551234","message":"¿Cuáles son los métodos de pago?"}'
```

Prueba de carga (20 órdenes simultáneas)

```
for i in $(seq 1 20); do
  curl -s -X POST http://localhost:5678/webhook/orden-nueva \
    -H "Content-Type: application/json" \
    -d "{\"customer_name\":\"Load Test $i\", \"customer_email\":\"test$i@test.com\", \"product_id\":2, \"quantity\":1}" &
done
wait
```

Verificar métricas

```
docker exec -it tesis_postgres psql -U n8n_user -d ecommerce_tesis \
  -c "SELECT * FROM v_metrics_summary;"
```

Ver emails enviados

Abrir http://localhost:8025 en el navegador

Ver dashboards

Abrir http://localhost:3000 en el navegador (admin / admin)

Anexo F: Lista de Figuras — Implementación de Desarrollo (SIMPLE)

Las siguientes figuras corresponden a los workflows activos y probados con datos reales en el entorno de desarrollo local.

Figura	Descripción	Sección
Figura 1	Canvas del Flujo 1 — Pipeline de Procesamiento de Órdenes en n8n (11 nodos, versión SIMPLE activa)	4.3.2
Figura 2	Canvas del Flujo 2 — Chatbot Omnicanal con IA en n8n (versión SIMPLE activa, webhook WhatsApp simulado)	4.4.2
Figura 3	Dashboard Grafana — “Pipeline Post-Venta Overview”: paneles MTDD, MTTR, distribución de estados. Vista en sección Grafana (4.5)	4.5
Figura 4	Mailpit — bandeja de emails capturados durante las pruebas funcionales (confirmaciones y avisos de sin stock)	4.6.2
Figura 5	Dashboard Grafana — “Pipeline Post-Venta Overview”: paneles MTDD (1.79 s), MTTR (7.69 s), distribución de estados de órdenes y evolución diaria. Vista en sección Resultados (5.1)	5.1
Figura 6	Dashboard Grafana — “Chatbot Omnicanal Métricas”: paneles TMR (2.38 s), intenciones detectadas y volumen diario por canal	5.2
Figura 7	Diagrama entidad-relación (ERD) de la base de datos ecommerce_tesis (5 tablas y sus relaciones), representado en formato ASCII en la Sección 4.2.2 del cuerpo del trabajo. La captura de pgAdmin se incluye como referencia visual complementaria.	4.2.2

Todas las capturas corresponden al entorno local de desarrollo. Los datos son simulados y representativos de un escenario típico de e-commerce.

Anexo G: Propuesta de Arquitectura de Producción

Las figuras A1 y A2 corresponden a los workflows de producción diseñados como extensión del sistema. Están disponibles en el repositorio como archivos JSON importables pero **no fueron ejecutados con datos reales** — representan la hoja de ruta técnica para una implementación productiva.

Figura	Descripción	Archivo en repositorio
Figura A1	Canvas del Flujo 1 PRODUCCIÓN: triggers WooCommerce, Shopify y nodo de normalización multi- plataforma	workflows/Flujo 1 – Pipeline de Procesamiento de Órdenes PRODUCCION.json
Figura A2	Canvas del Flujo 2 PRODUCCIÓN: triggers reales de Telegram Bot API, Gmail y WhatsApp Business Cloud API unificados con nodo Merge	workflows/Flujo 2 – Chatbot Omnicanal IA PRODUCCION.json

Figura A1: Canvas del Flujo 1 PRODUCCIÓN en n8n. Para importar: n8n → Workflows → Import from file.

Figura A2: Canvas del Flujo 2 PRODUCCIÓN en n8n. Requiere credenciales de Telegram Bot, Gmail y WhatsApp Business API para activarse.

Nota: Para reproducir el entorno completo ejecutar `docker compose up -d` en la raíz del repositorio.